UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

PROYECTO FINAL

Sistema de Facturación – Representaciones "Chapina", C.A.

Curso:

Algoritmos

Estudiante:

Werner Fernando Cifuentes Barillas

Carnet:

0901-25-23316

Fecha de Entrega:

14/10/2026

Índice

1. Introducción..... 3
2. Enunciado del proyecto..... 3
Catálogo de requerimientos..... 4
3. Planificación (Entregable 1)..... 5
3.1 Diagrama de Gantt..... 9
4. Planteamiento del problema (Entregable 2)..... 10
5. Análisis de requerimientos (Entregable 3)..... 12
6. Diagramas de caso de uso (Entregable 4)..... 16
7. Estudio de factibilidad y propuesta comercial (Entregables 5 y 6)..... 18
8. Manual de usuario y revisión de código (Entregable 7)..... 22
9. Plan de pruebas (Entregable 8)..... 37
10. Repositorio y ejecución del programa..... 52
11. Conclusiones..... 53
Anexo A. Código fuente (main.cpp)..... 54

1. Introducción

Este documento reúne el trabajo completo del proyecto final: el Sistema de Facturación de Representaciones "Chapina", C.A., una empresa dedicada a la distribución al por mayor de productos de limpieza. El sistema controla empleados (Vendedores y Repartidores), clientes, productos e inventario, facturas al contado y al crédito con IVA del 12 %, y pagos totales o parciales, además de emitir los seis reportes que necesita la gerencia.

El documento sigue el orden en que se desarrolló el proyecto: planificación, planteamiento del problema, análisis de requerimientos, casos de uso, factibilidad y propuesta comercial, manual de usuario con revisión de código, y plan de pruebas. Al final se incluyen las conclusiones y el código fuente del programa, desarrollado en C++17 con base de datos SQLite.

2. Enunciado del proyecto

Representaciones "Chapina", C.A. necesita un sistema de facturación que permita:

- Registrar los **empleados** de la empresa, diferenciando Vendedores y Repartidores, y asignar a cada Vendedor su cartera de clientes.
- Registrar los **clientes** con sus datos de contacto y su vendedor asignado.
- Registrar los **productos** con costo, precio y existencia, y controlar el inventario.
- Emitir **facturas** al contado o al crédito, con detalle de productos, IVA del 12 % y descuento automático del inventario.
- Registrar **pagos**, incluyendo pagos parciales, controlando el saldo de cada factura.
- Emitir seis **reportes**: Clientes, Vendedores, Productos Vendidos, Ventas del Día, Pagos del Día e Inventario Existente.

Toda la información debe almacenarse en una base de datos, con una entidad general de empleados que se especializa por tipo.

Catálogo de requerimientos

Resumen de los requerimientos del sistema. Los casos del plan de pruebas (Entregable 8) y la revisión de código (Entregable 7) se refieren a ellos por su número.

No.	Requerimiento	Tipo	Prioridad
REQ-0001	Gestión de empleados (Vendedores y Repartidores) y cartera de clientes de cada vendedor	Funcional	1 Alta
REQ-0002	Gestión de clientes e historial de facturas y pagos	Funcional	1 Alta
REQ-0003	Gestión de productos e inventario, con ingreso de mercadería	Funcional	1 Alta
REQ-0004	Facturación al contado y al crédito con detalle, IVA del 12 % y descuento de inventario	Funcional	1 Alta
REQ-0005	Registro de pagos totales y parciales con control de saldo	Funcional	1 Alta
REQ-0006	Seis reportes: Clientes, Vendedores, Productos Vendidos, Ventas del Día, Pagos del Día e Inventario	Funcional	1 Alta
REQ-0007	Interfaz sencilla, con menús y mensajes claros	No funcional	2 Media
REQ-0008	Rendimiento y disponibilidad: respuesta rápida y datos que no se pierden	No funcional	2 Media
REQ-0009	Autenticación por rol y validación de datos	No funcional	1 Alta
REQ-0010	Posibilidad de crecimiento y uso en los equipos actuales de la empresa	No funcional	3 Baja

Sistema de Facturación

Representaciones "Chapina", C.A.

Elaborado por: Werner Cifuentes

1. Planificación

1.1 Responsable del proyecto

Nombre	Rol / Responsabilidad
Werner Cifuentes	Encargado único del proyecto: análisis, diseño, documentación y desarrollo

1.2 Objetivo general del proyecto

Desarrollar un sistema de información que automatice el proceso de facturación, control de clientes, vendedores, inventario y pagos de Representaciones "Chapina", C.A., empresa dedicada a la venta al mayor de artículos de limpieza, con el fin de sustituir el manejo manual actual y generar de forma ágil y confiable los reportes que la gerencia requiere para la toma de decisiones.

1.3 Objetivos específicos

- Diseñar una base de datos que almacene de forma estructurada la información de vendedores, clientes, productos, facturas y pagos.
- Implementar el módulo de facturación que permita registrar ventas al contado y al crédito, asociando cada factura a un vendedor y a un cliente.
- Controlar el inventario de productos, actualizando existencias conforme se generan las ventas.
- Registrar los pagos realizados por los clientes a cada factura, incluyendo pagos parciales para las ventas a crédito.
- Generar los seis reportes requeridos por la empresa: Clientes, Vendedores, Productos Vendidos, Ventas del Día, Pagos del Día e Inventario Existente.
- Entregar la documentación completa del proyecto (análisis, diseño, factibilidad, manual de usuario y plan de pruebas) conforme al calendario establecido.

1.4 Alcance del proyecto

El proyecto abarca el análisis, diseño y documentación de un sistema de facturación para Representaciones "Chapina", C.A. Incluye la definición de requerimientos, el modelado de casos de uso, el estudio de factibilidad, la propuesta comercial, el manual de usuario y el plan de pruebas. El sistema contemplará los módulos de Vendedores, Clientes, Productos/Inventario, Facturación y Pagos, así como la generación de los reportes solicitados por la empresa.

1.5 Cronograma de actividades (Diagrama de Gantt)

El proyecto se organiza en cinco fases: Análisis de Requerimientos, Diseño, Desarrollo, Implementación, y Mantenimiento y Pruebas. A continuación se detalla el cronograma de la Fase 1, que corresponde al análisis, y el resumen general de las fases siguientes:

Fase / Actividad	Fecha inicio	Fecha fin
Fase 1 - Análisis de Requerimientos	19/08/2026	25/09/2026
Recopilación de información	19/08/2026	25/08/2026
Procesamiento de información	26/08/2026	28/08/2026
Generar documento de requerimientos	31/08/2026	02/09/2026
Planteamiento del problema	31/08/2026	01/09/2026
Estudio de factibilidad	31/08/2026	11/09/2026
Propuesta comercial	14/09/2026	25/09/2026
Fase 2 - Diseño (Casos de uso, ER)	19/08/2026	19/08/2026
Fase 3 - Desarrollo (Módulos del sistema)	19/08/2026	19/08/2026
Fase 4 - Implementación (Despliegue, manuales)	19/08/2026	19/08/2026
Fase 5 - Mantenimiento y Pruebas	19/08/2026	19/08/2026

1.6 Calendario de entregas

Las entregas parciales del proyecto se realizarán conforme al siguiente calendario:

No.	Entregable	Fecha
1	Planificación	26/08/2026
2	Planteamiento del problema	02/09/2026
3	Análisis de requerimientos	09/09/2026
4	Diagramas de caso de uso	16/09/2026
5	Estudio de factibilidad – técnica, operativa, económica y legal	30/09/2026
6	Propuesta comercial/económica	30/09/2026
7	Manual de usuario – Revisión de código	07/10/2026
8	Plan de pruebas	14/10/2026

2. Planteamiento del problema

2.1 Situación actual

Representaciones "Chapina", C.A. es una empresa dedicada a la venta al mayor de artículos de limpieza a clientes minoristas, bajo la modalidad de pagos a crédito o a contado. Actualmente, la empresa cuenta con un grupo de vendedores, cada uno con una cartera fija de clientes asignados, y el proceso de facturación, control de pagos e inventario se realiza sin el apoyo de un sistema integrado que centralice esta información.

Esta situación provoca que datos clave de la operación diaria —clientes, vendedores, productos en existencia, facturas emitidas y pagos recibidos— se manejen de forma dispersa, lo que dificulta mantener actualizado el saldo de cada factura a crédito, conocer con exactitud las existencias de producto y generar con rapidez los reportes que la gerencia necesita para dar seguimiento al negocio.

2.2 Definición del problema

La empresa no cuenta con un sistema que integre el registro de vendedores, clientes, productos, facturación y pagos, ni que permita generar de manera automática los reportes de Clientes, Vendedores, Productos Vendidos, Ventas del Día, Pagos del Día e Inventario Existente. Esto genera retrasos en la obtención de información, riesgo de errores en el cálculo de saldos de facturas a crédito, y falta de control sobre las existencias reales de productos, lo que puede derivar en pérdida de ventas por desabastecimiento o en descuadres en la cartera de cobro.

2.3 Justificación

Un sistema de facturación automatizado permitirá a Representaciones "Chapina", C.A. registrar de forma ordenada cada venta realizada por sus vendedores, mantener el control exacto de los pagos —incluyendo pagos parciales en ventas a crédito— y conocer en todo momento la existencia real de productos. Además, la generación automática de los reportes solicitados reducirá el tiempo que actualmente se invierte en consolidar esta información manualmente, mejorando la toma de decisiones comerciales y el control administrativo de la empresa.

2.4 Objetivo general

Resolver la falta de control centralizado sobre la facturación, los pagos y el inventario de Representaciones "Chapina", C.A., mediante el desarrollo de un sistema que registre y relacione la información de vendedores, clientes, productos y transacciones, y que genere de forma automática los reportes requeridos por la gerencia.

2.5 Objetivos específicos

- Eliminar el manejo disperso de la información de clientes, vendedores y productos.
- Garantizar el cálculo correcto del saldo pendiente en facturas a crédito con múltiples pagos.
- Mantener actualizado en tiempo real el inventario de productos conforme a las ventas realizadas.

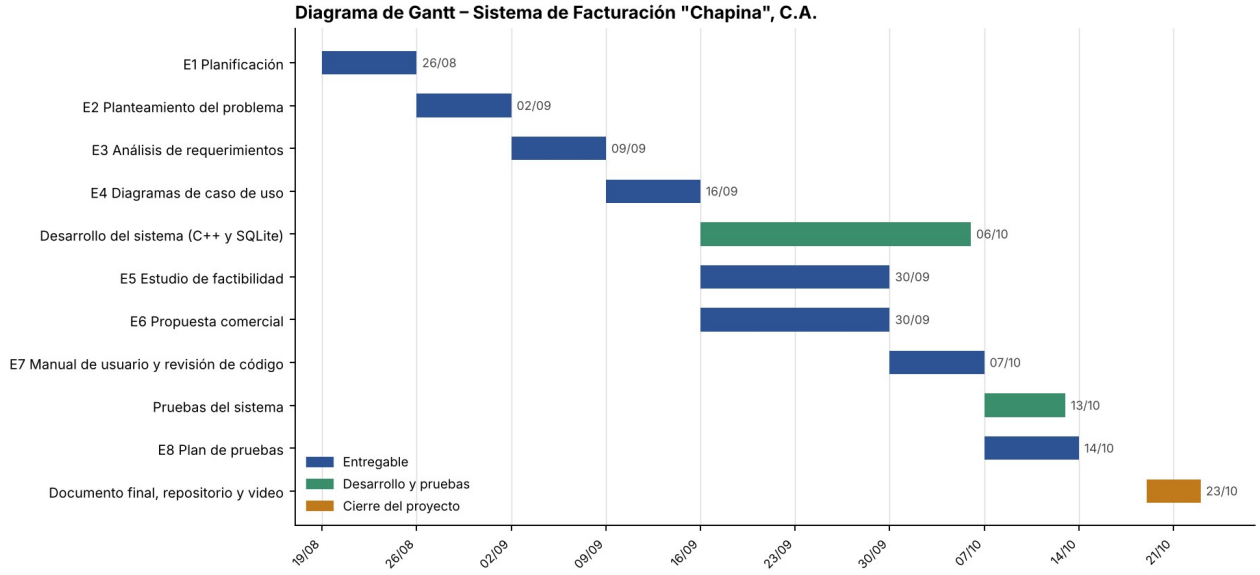
- Reducir el tiempo de generación de los reportes de Clientes, Vendedores, Productos Vendidos, Ventas del Día, Pagos del Día e Inventario Existente.

2.6 Alcance de la solución

La solución propuesta abarcará el registro de vendedores (código de empleado, DPI, nombre, teléfono, dirección, e-mail) y clientes (nombre, NIT, teléfono, dirección, persona de contacto, e-mail, página web), la emisión de facturas con su detalle de productos vendidos, el registro de pagos asociados a cada factura, el control de inventario de productos (código, nombre, costo, precio, existencia), y la generación de los seis reportes definidos por la empresa. No incluye, en esta etapa, integración con sistemas contables externos ni facturación electrónica ante la administración tributaria, lo cual podrá evaluarse en una fase posterior del proyecto.

3.1 Diagrama de Gantt del proyecto

El diagrama muestra las fechas reales del proyecto: cada entregable con su semana de trabajo, el desarrollo del sistema y sus pruebas, y el cierre con el documento final, el repositorio y el video.



Las barras azules son los entregables del calendario del curso, las verdes el desarrollo y las pruebas del sistema, y la naranja el cierre del proyecto. El desarrollo del programa se hizo en paralelo con el estudio de factibilidad y la propuesta comercial, y las pruebas se realizaron la misma semana del Plan de pruebas.

Planteamiento del Problema

1. Situación actual

Representaciones "Chapina", C.A. es una empresa dedicada a la venta al mayor de artículos de limpieza a clientes minoristas, bajo la modalidad de pagos a crédito o a contado. Actualmente, la empresa cuenta con un grupo de vendedores, cada uno con una cartera fija de clientes asignados, y el proceso de facturación, control de pagos e inventario se realiza sin el apoyo de un sistema integrado que centralice esta información.

Esta situación provoca que datos clave de la operación diaria —clientes, vendedores, productos en existencia, facturas emitidas y pagos recibidos— se manejen de forma dispersa, lo que dificulta mantener actualizado el saldo de cada factura a crédito, conocer con exactitud las existencias de producto y generar con rapidez los reportes que la gerencia necesita para dar seguimiento al negocio.

2. Definición del problema

La empresa no cuenta con un sistema que integre el registro de vendedores, clientes, productos, facturación y pagos, ni que permita generar de manera automática los reportes de Clientes, Vendedores, Productos Vendidos, Ventas del Día, Pagos del Día e Inventario Existente. Esto genera retrasos en la obtención de información, riesgo de errores en el cálculo de saldos de facturas a crédito, y falta de control sobre las existencias reales de productos, lo que puede derivar en pérdida de ventas por desabastecimiento o en descuadres en la cartera de cobro.

3. Justificación

Un sistema de facturación automatizado permitirá a Representaciones "Chapina", C.A. registrar de forma ordenada cada venta realizada por sus vendedores, mantener el control exacto de los pagos —incluyendo pagos parciales en ventas a crédito— y conocer en todo momento la existencia real de productos. Además, la generación automática de los reportes solicitados reducirá el tiempo que actualmente se invierte en consolidar esta información manualmente, mejorando la toma de decisiones comerciales y el control administrativo de la empresa.

4. Objetivo general

Resolver la falta de control centralizado sobre la facturación, los pagos y el inventario de Representaciones "Chapina", C.A., mediante el desarrollo de un sistema que registre y relacione la información de vendedores, clientes, productos y transacciones, y que genere de forma automática los reportes requeridos por la gerencia.

5. Objetivos específicos

- Eliminar el manejo disperso de la información de clientes, vendedores y productos.
- Garantizar el cálculo correcto del saldo pendiente en facturas a crédito con múltiples pagos.
- Mantener actualizado en tiempo real el inventario de productos conforme a las ventas realizadas.
- Reducir el tiempo de generación de los reportes de Clientes, Vendedores, Productos Vendidos, Ventas del Día, Pagos del Día e Inventario Existente.

6. Alcance de la solución

La solución propuesta abarcará el registro de vendedores (código de empleado, DPI, nombre, teléfono, dirección, e-mail) y clientes (nombre, NIT, teléfono, dirección, persona de contacto, e-mail, página web), la emisión de facturas con su detalle de productos vendidos, el registro de pagos asociados a cada factura, el control de inventario de productos (código, nombre, costo, precio, existencia), y la generación de los seis reportes definidos por la empresa. No incluye, en esta etapa, integración con sistemas contables externos ni facturación electrónica ante la administración tributaria, lo cual podrá evaluarse en una fase posterior del proyecto.

Análisis de Requerimientos

1. Introducción

El presente análisis identifica los requerimientos funcionales y no funcionales del Sistema de Facturación para Representaciones "Chapina", C.A., a partir del enunciado del proyecto. Estos requerimientos servirán de base para el diseño de los casos de uso, el modelo de datos y el desarrollo del sistema.

2. Requerimientos funcionales

2.1 Gestión de Empleados (Vendedores y Repartidores)

Código	Requerimiento	Descripción
RF-01	Registro de empleados (generalización)	El sistema debe permitir registrar, modificar y consultar empleados con: código de empleado, DPI, nombre, teléfono, dirección, e-mail y tipo de empleado (Vendedor o Repartidor). Vendedor y Repartidor se generalizan en la entidad Empleado; el Repartidor no tiene, por ahora, atributos propios adicionales.
RF-02	Asignación de clientes	El sistema debe permitir asociar a cada empleado de tipo Vendedor la cartera de clientes fijos que tiene a su cargo.

2.2 Gestión de Clientes

Código	Requerimiento	Descripción
RF-03	Registro de clientes	El sistema debe permitir registrar, modificar y consultar clientes con: nombre, NIT, teléfono, dirección, persona de contacto, e-mail y página web.
RF-04	Consulta de historial de cliente	El sistema debe permitir consultar las facturas y pagos asociados a un cliente específico.

2.3 Gestión de Productos e Inventario

Código	Requerimiento	Descripción
RF-05	Registro de productos	El sistema debe permitir registrar, modificar y consultar productos con: código, nombre, costo, precio y existencia.
RF-06	Control de inventario	El sistema debe descontar automáticamente la existencia de cada producto al momento de generarse una factura.
RF-07	Alerta de existencia	El sistema debe evitar la venta de productos cuya existencia sea insuficiente para cubrir la cantidad

Código	Requerimiento	Descripción
		solicitada.

2.4 Facturación

Código	Requerimiento	Descripción
RF-08	Generación de factura	El sistema debe permitir generar una factura asociada a un vendedor y a un cliente, registrando: número correlativo, número de serie, total, IVA, subtotal, condición (contado/crédito), plazo y fecha de vencimiento.
RF-09	Detalle de factura	El sistema debe registrar el detalle de productos vendidos en cada factura (producto, cantidad, precio unitario y subtotal por línea).
RF-10	Cálculo automático	El sistema debe calcular automáticamente el subtotal, el IVA y el total de cada factura a partir del detalle de productos.

2.5 Pagos

Código	Requerimiento	Descripción
RF-11	Registro de pagos	El sistema debe permitir registrar pagos realizados por los clientes a una factura, guardando número correlativo, fecha, monto y saldo.
RF-12	Pago único para facturas al contado	Cuando la factura sea al contado, el sistema debe permitir un único pago asociado que cubra el total de la factura.
RF-13	Pagos múltiples para facturas a crédito	Cuando la factura sea a crédito, el sistema debe permitir el registro de varios pagos parciales, actualizando el saldo pendiente después de cada pago.
RF-14	Control de saldo	El sistema debe calcular y mostrar en todo momento el saldo pendiente de cada factura a crédito.

2.6 Reportes

Código	Requerimiento	Descripción
RF-15	Reporte de Clientes	El sistema debe generar un listado de todos los clientes registrados con sus datos generales.
RF-16	Reporte de Vendedores	El sistema debe generar un listado de todos los vendedores registrados con sus datos generales.
RF-17	Reporte de Productos Vendidos	El sistema debe generar un listado de los productos vendidos en un periodo determinado.

Código	Requerimiento	Descripción
RF-18	Listado de Ventas (Facturas del Día)	El sistema debe generar un listado de las facturas emitidas en la fecha del día.
RF-19	Reporte de Pagos del Día	El sistema debe generar un listado de los pagos recibidos en la fecha del día.
RF-20	Reporte de Inventario Existente	El sistema debe generar un listado del inventario actual de productos con su existencia disponible.

3. Requerimientos no funcionales

Código	Requerimiento	Descripción
RNF-01	Usabilidad	La interfaz debe ser sencilla e intuitiva para que los vendedores puedan registrar facturas sin necesidad de capacitación extensa.
RNF-02	Rendimiento	El sistema debe generar cualquiera de los reportes solicitados en un tiempo no mayor a unos pocos segundos, aun con un volumen considerable de facturas.
RNF-03	Disponibilidad	El sistema debe estar disponible durante el horario laboral de la empresa para el registro de ventas y pagos.
RNF-04	Seguridad	El acceso al sistema debe requerir autenticación, y solo usuarios autorizados podrán modificar datos de productos, precios y existencias.
RNF-05	Integridad de datos	El sistema debe validar que no se generen facturas ni pagos con datos incompletos o inconsistentes (por ejemplo, montos negativos).
RNF-06	Escalabilidad	El sistema debe permitir el crecimiento futuro en número de clientes, vendedores y productos sin degradar su funcionamiento.
RNF-07	Compatibilidad	El sistema debe ser accesible desde los equipos con los que actualmente cuenta la empresa.

4. Requerimientos de datos (entidades principales)

A partir de los requerimientos funcionales se identifican las siguientes entidades de información que el sistema deberá almacenar:

Entidad	Atributos
Empleado	Código de empleado, DPI, Nombre, Teléfono, Dirección, E-mail, Tipo de empleado (Vendedor / Repartidor)
Cliente	Nombre, NIT, Teléfono, Dirección, Persona de contacto, E-mail, Página web
Producto	Código, Nombre, Costo, Precio, Existencia

Entidad	Atributos
Factura	N° Correlativo, N° de serie, Total, IVA, Subtotal, Condición (contado/crédito), Plazo, Fecha de vencimiento, Vendedor, Cliente
Detalle de Factura	Factura, Producto, Cantidad, Precio unitario, Subtotal
Pago	N° Correlativo, Fecha, Monto, Saldo, Factura asociada

5. Requerimientos de reportes

El sistema debe generar los siguientes seis reportes solicitados por la empresa:

Reporte	Contenido
Reporte de Clientes	Listado de todos los clientes con nombre, NIT, teléfono, dirección, persona de contacto, e-mail y página web.
Reporte de Vendedores	Listado de todos los vendedores con código de empleado, DPI, nombre, teléfono, dirección y e-mail.
Reporte de Productos Vendidos	Listado de productos vendidos en un periodo, con cantidad total vendida por producto.
Listado de Ventas (Facturas del Día)	Listado de las facturas emitidas en la fecha actual, con cliente, vendedor, condición y total.
Reporte de Pagos del Día	Listado de los pagos recibidos en la fecha actual, con factura asociada, monto y saldo restante.
Reporte de Inventario Existente	Listado de todos los productos con su existencia actual, costo y precio.

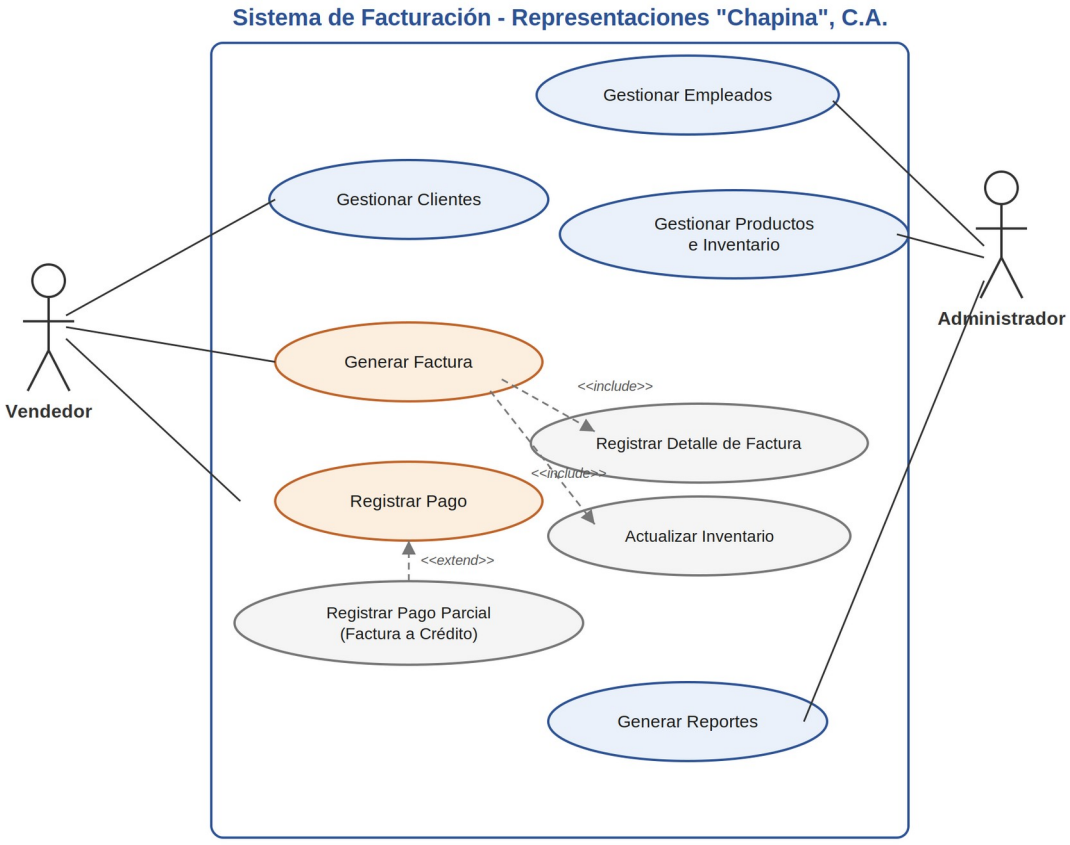
Diagrama de Casos de Uso

1. Actores

Vendedor: empleado encargado de atender a los clientes asignados, generar facturas y registrar los pagos que estos realizan.

Administrador: encargado de la gestión general del negocio, incluyendo el registro de empleados (Vendedores y Repartidores), el control de productos e inventario, y la obtención de los reportes gerenciales.

2. Diagrama



3. Descripción de los casos de uso

Caso de uso	Actor(es)	Descripción
Gestionar Empleados	Administrador	Permite registrar, modificar y consultar empleados (Vendedores y Repartidores) con sus datos generales.
Gestionar Clientes	Vendedor	Permite registrar, modificar y consultar clientes, así como su cartera asignada y su historial de facturas y pagos.
Gestionar Productos e Inventario	Administrador	Permite registrar, modificar y consultar productos, controlando su existencia disponible.
Generar Factura	Vendedor	Permite generar una factura para un cliente, asociada al vendedor que realiza la venta, incluyendo el detalle de productos vendidos.
Registrar Detalle de Factura	Vendedor	Caso de uso incluido por Generar Factura: registra cada producto vendido, cantidad y precio dentro de la factura.
Actualizar Inventario	Vendedor	Caso de uso incluido por Generar Factura: descuenta automáticamente del inventario la cantidad de productos vendidos.
Registrar Pago	Vendedor	Permite registrar un pago realizado por el cliente sobre una factura, actualizando su saldo.
Registrar Pago Parcial (Factura a Crédito)	Vendedor	Extiende a Registrar Pago: permite registrar varios pagos parciales sobre una misma factura cuando esta es a crédito.
Generar Reportes	Administrador	Permite generar los reportes de Clientes, Vendedores, Productos Vendidos, Ventas del Día, Pagos del Día e Inventario Existente.

Parte 1: Estudio de Factibilidad

1. Introducción

El presente estudio evalúa la viabilidad del Sistema de Facturación para Representaciones "Chapina", C.A. desde cuatro perspectivas: técnica, operativa, económica y legal, con el fin de determinar si el proyecto es factible de ejecutar en las condiciones y el tiempo establecidos.

2. Factibilidad Técnica

Aspecto	Detalle
Lenguaje de programación	C++ (estándar C++17), aplicación de consola.
Base de datos	SQLite 3 (motor embebido, un solo archivo .db, sin servidor dedicado).
Herramientas de desarrollo	Compilador g++ / GCC y librería sqlite3, ambos de uso libre y multiplataforma.
Requerimientos de hardware	Cualquier computadora de oficina estándar (procesador de gama básica, 4 GB de RAM o más); no se requiere equipo especializado.
Conocimiento técnico necesario	Programación en C++ y SQL básico, con los que ya se cuenta para el desarrollo del proyecto.
Conclusión técnica	El proyecto es técnicamente factible: la tecnología elegida es gratuita, liviana, no requiere infraestructura adicional y es adecuada para el volumen de datos de la empresa.

3. Factibilidad Operativa

Aspecto	Detalle
Usuarios del sistema	Vendedores (registro de clientes, facturación y pagos) y el Administrador (gestión de empleados, productos y reportes).
Capacitación requerida	Mínima: al ser un menú de opciones numeradas, no se requiere capacitación técnica avanzada, solo una inducción breve al flujo de cada módulo.
Resistencia al cambio	Baja. El sistema sustituye un proceso manual, por lo que el personal percibirá una reducción de trabajo repetitivo (cálculos de IVA, saldos y reportes).
Disponibilidad operativa	El sistema puede operar en cualquier equipo de la empresa sin depender de conexión a internet, ya que la base de datos es local.
Conclusión operativa	El proyecto es operativamente factible: se ajusta al flujo de trabajo actual de la empresa y no exige cambios significativos en la forma de operar del personal.

4. Factibilidad Económica

A continuación se presenta una estimación de los costos asociados al proyecto. El detalle completo de la inversión y forma de pago se desarrolla en la Parte 2 (Propuesta Comercial/Económica).

Concepto	Costo estimado (Q.)	Tipo
Desarrollo del sistema (análisis, diseño, programación y pruebas)	28,800.00	Único

Concepto	Costo estimado (Q.)	Tipo
Compilador C++ (GCC) y motor SQLite	0.00	N/A (software libre)
Equipo de cómputo	0.00	N/A (se usa el existente)
Capacitación al personal	800.00	Único
Soporte y mantenimiento (mensual)	600.00	Recurrente

La inversión inicial se recupera mediante la reducción de tiempo invertido en tareas manuales (cálculo de totales, control de saldos e inventario) y la disminución de errores en la facturación, lo cual se traduce en una mejor gestión de cobros y de existencias. Se considera económicamente factible dado que el costo es proporcional al beneficio operativo esperado.

5. Factibilidad Legal

Aspecto	Detalle
Licenciamiento de software	GCC y SQLite se distribuyen bajo licencias libres (GPL y dominio público, respectivamente), por lo que no existe restricción legal para su uso comercial.
Propiedad del sistema	El código desarrollado será propiedad de Representaciones "Chapina", C.A. una vez entregado y aceptado formalmente.
Protección de datos	El sistema almacena datos de clientes (nombre, NIT, dirección) que deben resguardarse con los controles de acceso definidos en los requerimientos no funcionales (RNF de seguridad).
Cumplimiento fiscal	El sistema calcula y registra el IVA en cada factura; no sustituye, en esta primera versión, la facturación electrónica ante la SAT, lo cual queda fuera del alcance actual.
Conclusión legal	El proyecto es legalmente factible: no existen impedimentos de licenciamiento y se puede operar dentro del marco normativo vigente, dejando documentado lo que queda fuera de alcance.

6. Conclusión general

Con base en el análisis anterior, el proyecto es factible en sus cuatro dimensiones: cuenta con tecnología adecuada y de bajo costo (técnica), se ajusta al flujo de trabajo actual de la empresa (operativa), su inversión es proporcional al beneficio esperado (económica), y no presenta impedimentos de licenciamiento o cumplimiento normativo (legal). Se recomienda continuar con el desarrollo del sistema conforme al cronograma establecido.

Parte 2: Propuesta Comercial/Económica

1. Presentación

Se presenta a Representaciones "Chapina", C.A. la propuesta para el desarrollo del Sistema de Facturación, orientado a automatizar el registro de empleados, clientes, productos, facturación y pagos, así como la generación de los reportes requeridos por la gerencia. La propuesta se basa en el análisis de requerimientos y los casos de uso ya levantados para el proyecto.

2. Alcance del servicio

Módulo	Incluye
Empleados	Registro, modificación y consulta de empleados (Vendedores y Repartidores).
Clientes	Registro, modificación y consulta de clientes, asignación a vendedor y consulta de historial.
Productos e Inventario	Registro de productos, control de existencias y descuento automático al facturar.
Facturación	Generación de facturas al contado y al crédito, con cálculo automático de IVA, subtotal y total.
Pagos	Registro de pagos, incluyendo pagos parciales para facturas a crédito, con control de saldo.
Reportes	Clientes, Vendedores, Productos Vendidos, Ventas del Día, Pagos del Día e Inventario Existente.

Fuera de alcance de esta propuesta
Facturación electrónica ante la SAT (FEL) e integración con sistemas contables externos. Podrá cotizarse como una fase adicional.

3. Cronograma de entrega

Etapa	Inicio	Fin
Análisis y diseño (requerimientos, casos de uso, modelo de datos)	19/08/2026	16/09/2026
Estudio de factibilidad y propuesta comercial	17/09/2026	30/09/2026
Desarrollo del sistema (C++ / SQLite)	01/10/2026	06/10/2026
Manual de usuario y revisión de código	07/10/2026	07/10/2026
Plan de pruebas y entrega final	08/10/2026	14/10/2026

4. Inversión

El costo de desarrollo se calculó con base en las horas estimadas por etapa y una tarifa de mercado de Q180.00 por hora para desarrollo de software a medida:

Etapa	Horas	Tarifa/hora	Subtotal
Análisis y diseño (requerimientos, casos de uso, modelo de datos)	30	Q 180.00	Q 5,400.00
Desarrollo (6 módulos en C++ con SQLite)	100	Q 180.00	Q 18,000.00
Pruebas y depuración	20	Q 180.00	Q 3,600.00
Documentación (manual de usuario, plan de pruebas)	10	Q 180.00	Q 1,800.00
TOTAL	**160**		**Q 28,800.00**

Concepto	Costo (Q.)	Tipo
Análisis, diseño, desarrollo y pruebas del sistema	28,800.00	Único
Capacitación al personal (vendedores y administrador)	800.00	Único
Soporte y mantenimiento correctivo	600.00	Mensual*
Software base (GCC, SQLite)	0.00	Sin costo
TOTAL INVERSIÓN INICIAL	**Q 29,600.00**	

* El soporte mensual es opcional, a partir del mes siguiente a la entrega final del sistema.

5. Forma de pago

Pago	Condición
Primer pago (40%)	Q 11,840.00 al aceptar la propuesta, para iniciar el desarrollo.
Segundo pago (40%)	Q 11,840.00 contra entrega del sistema funcional y el manual de usuario (07/10/2026).
Pago final (20%)	Q 5,920.00 contra entrega del plan de pruebas y aceptación final del sistema (14/10/2026).

6. Condiciones generales

- La propuesta tiene una vigencia de 15 días calendario a partir de la fecha de entrega.
- El código fuente y la base de datos del sistema serán propiedad de Representaciones "Chapina", C.A. una vez cancelado el pago final.
- Cualquier requerimiento fuera del alcance descrito en la sección 2 será cotizado por separado.
- El soporte y mantenimiento no incluye el desarrollo de nuevos módulos.

1. Introducción

El Sistema de Facturación de Representaciones "Chapina", C.A. es una aplicación de consola desarrollada en C++ con base de datos SQLite. Automatiza el trabajo diario de la empresa: registro de empleados (Vendedores y Repartidores), clientes, productos e inventario, emisión de facturas al contado y al crédito, control de pagos y generación de los seis reportes que solicita la gerencia.

Este documento tiene dos partes. La **Parte 1** es el manual de usuario: explica paso a paso cómo poner en marcha y usar el sistema. La **Parte 2** es la revisión de código: describe cómo está construido el programa, su modelo de datos y cómo se relaciona con los requerimientos documentados.

Todas las figuras son salidas reales del programa, obtenidas al ejecutarlo con datos de ejemplo el 07/10/2026.

PARTE 1: MANUAL DE USUARIO

2. Requisitos e instalación

Componente	Requisito
Sistema operativo	Windows 10 o superior, Linux o macOS
Compilador	g++ con soporte para C++17 (en Windows, MinGW-w64)
Librería de base de datos	SQLite 3 (en Linux: paquete `libsqlite3-dev`)
Espacio en disco	Menos de 5 MB
Archivos del sistema	`main.cpp` (código fuente) y `chapina.db` (base de datos; se crea sola si no existe)

2.1 Compilar

Abra una terminal en la carpeta donde está `main.cpp` y ejecute:

```
# Linux / macOS
g++ -std=c++17 -o chapina main.cpp -lsqlite3

# Windows (con el archivo sqlite3.c descargado de sqlite.org)
gcc -c sqlite3.c
g++ -std=c++17 -o chapina.exe main.cpp sqlite3.o
```

2.2 Ejecutar

Ejecute `./chapina` (en Windows, `chapina.exe`). Si el archivo `chapina.db` no existe, el sistema lo crea automáticamente con todas las tablas vacías. Para trabajar con otra base de datos, por ejemplo la de demostración que incluye datos de ejemplo, indíquela al ejecutar:

```
./chapina chapina_demo.db
```

Para respaldar la información basta con copiar el archivo `.db`.

3. Acceso al sistema

Al iniciar, el sistema solicita usuario y clave; después de tres intentos fallidos se cierra. Según el perfil, se habilitan unas opciones u otras:

Perfil	Usuario	Clave	Opciones disponibles
Administrador	`admin`	`admin123`	Empleados, Clientes, Productos e Inventario, Facturación, Pagos y Reportes
Vendedor	Su código de empleado (ej. `V001`)	Su DPI	Clientes, Facturación y Pagos, únicamente de los clientes de su cartera

Los usuarios Vendedor son los empleados de tipo "Vendedor" registrados en el módulo de Empleados. Los Repartidores no ingresan al sistema.

Figura 1. Inicio de sesión como Administrador y menú principal.

```
======================================================
  SISTEMA DE FACTURACIÓN - REPRESENTACIONES "CHAPINA"
======================================================
Usuario (admin o código de empleado): admin
Clave: admin123

Bienvenido(a), Administrador

========== MENÚ PRINCIPAL ==========
  1. Empleados
  2. Clientes
  3. Productos e Inventario
  4. Facturación
  5. Pagos
  6. Reportes
  0. Salir del sistema
Seleccione una opción: 1
```

Navegación. Escriba el número de la opción y presione Enter. La opción 0 regresa al menú anterior (en el menú principal, cierra el sistema). En los datos opcionales basta con presionar Enter para dejarlos en blanco; al modificar un registro, Enter conserva el valor actual que aparece entre corchetes.

4. Uso de los módulos

4.1 Empleados (Administrador)

Todo el personal se registra en una sola entidad **Empleado**, que se distingue por su tipo: **Vendedor** o **Repartidor**.

1. Menú principal → **1. Empleados** → **1. Registrar empleado**.
2. Complete código de empleado, DPI, nombre, teléfono, dirección y e-mail.
3. Elija el tipo: 1 = Vendedor, 2 = Repartidor.

Figura 2. Registro de un empleado.

```
--- Registrar empleado ---
Código de empleado : V001
DPI                : 2456789010101
Nombre             : Carlos Méndez López
Teléfono           : 5512-3456
Dirección          : Zona 5, Guatemala
E-mail             : cmendez@chapina.com
Tipo (1=Vendedor, 2=Repartidor): 1
Empleado registrado correctamente.
```

Figura 3. Consulta de empleados (opción 3).

```
--- Empleados registrados ---
-------------------------------------------------------------------------------------------------
Código    DPI             Nombre                  Teléfono    E-mail                  Tipo
-------------------------------------------------------------------------------------------------
R001      3011223340101   Pedro Sicán             5534-9900   psican@chapina.com      Repartidor
V001      2456789010101   Carlos Méndez López     5512-3456   cmendez@chapina.com     Vendedor
V002      2987654320101   Ana Lucía Ruiz          5523-7788   aruiz@chapina.com       Vendedor
-------------------------------------------------------------------------------------------------
Total de registros: 3
```

El código de empleado y el DPI no pueden repetirse, y el tipo no se puede cambiar una vez registrado. La opción 2 (Modificar) permite corregir nombre, teléfono, dirección y e-mail.

4.2 Clientes

Cada cliente pertenece a un Vendedor (su cartera fija). Si quien registra es un Vendedor, el cliente queda asignado a él automáticamente; si es el Administrador, debe indicar el código del vendedor. El NIT no puede repetirse.

Figura 4. Registro de un cliente por un Vendedor.

```
--- Registrar cliente ---
Nombre           : Mini Super San José
NIT              : 9988776-K
Teléfono         : 2255-0099
Dirección        : Avenida Petapa, Zona 12
Persona contacto : Rosa Díaz
E-mail           : minisuper@gmail.com
Página web       :
Vendedor asignado: V001 (usted)
Cliente registrado correctamente.
```

La opción 4 (**Historial de un cliente**) muestra sus facturas con el saldo de cada una y todos los pagos que ha realizado:

Figura 5. Historial de un cliente: facturas y pagos.

```
Cliente: Mini Super San José   NIT: 9988776-K

Facturas:
-------------------------------------------------------------
 Factura  Fecha       Condición          Total         Saldo
-------------------------------------------------------------
       2  2026-10-07  Crédito          1205.12        305.12
-------------------------------------------------------------
Total de registros: 1

Pagos realizados:
-----------------------------------------------------------
  Pago   Factura  Fecha              Monto         Saldo
-----------------------------------------------------------
     2         2  2026-10-07        500.00        705.12
     3         2  2026-10-07        400.00        305.12
-----------------------------------------------------------
Total de registros: 2
```

4.3 Productos e inventario (Administrador)

Registre cada producto con código, nombre, costo, precio y existencia inicial; el sistema avisa si el precio es menor al costo. Cuando llega mercadería, use **Ingreso de mercadería** para sumar unidades. Las ventas descuentan la existencia automáticamente.

Figura 6. Registro de un producto.

```
--- Registrar producto ---
Código     : P001
Nombre     : Jabón líquido multiusos 1L
Costo (Q)  : 18.50
Precio (Q) : 25.00
Existencia : 120
Producto registrado correctamente.
```

Figura 7. Ingreso de mercadería al inventario.

```
--- Ingreso de mercadería (inventario) ---
------------------------------------------------------------------------
Código    Producto                          Costo      Precio  Existencia
------------------------------------------------------------------------
P001      Jabón líquido multiusos 1L         18.50       25.00         120
P002      Desinfectante pino 1 galón         32.00       48.00          60
P003      Cloro 1 galón                      15.00       22.50          80
P004      Escoba plástica                    20.00       30.00          35
------------------------------------------------------------------------
Total de registros: 4
Código del producto: P004
Cantidad que ingresa a bodega: 15
Existencia actualizada: Escoba plástica ahora tiene 50 unidades.
```

4.4 Facturación

1. Menú principal → **Facturación** → **1. Generar factura**.
2. Elija el cliente por su ID.
3. Seleccione la condición: 1 = Contado, 2 = Crédito. Si es crédito, indique el plazo en días.
4. Escriba el código de cada producto y la cantidad. Repita las veces necesarias y presione Enter sin código para terminar.
5. Revise el resumen (subtotal, IVA 12 % y total) y confirme con **S**.

La factura se asigna al vendedor del cliente, el IVA y el total se calculan solos, y al confirmar se descuenta el inventario. Una factura **al contado** queda pagada de inmediato con un único pago por el total; una factura **a crédito** queda con saldo igual al total y con fecha de vencimiento igual a la fecha de emisión más el plazo.

Figura 8. Factura al contado (se omite la lista de productos disponibles).

```
ID del cliente: 1
Cliente: Tienda La Esperanza  |  Vendedor asignado: V001
Condición (1=Contado, 2=Crédito): 1
      [...]
Código de producto (Enter para terminar): P001
Cantidad (disponible 120): 10
  Agregado: 10 x Jabón líquido multiusos 1L a Q 25.00 c/u
Código de producto (Enter para terminar): P004
Cantidad (disponible 50): 100
  Existencia insuficiente. Solo hay 50 unidades disponibles.
Código de producto (Enter para terminar): P003
Cantidad (disponible 80): 5
  Agregado: 5 x Cloro 1 galón a Q 24.00 c/u
Código de producto (Enter para terminar):

Resumen: Subtotal Q 370.00 | IVA Q 44.40 | TOTAL Q 414.40
¿Confirmar la factura? (S/N): S

Factura generada correctamente.

==================== FACTURA ====================
No. correlativo : 1     Serie: A
Fecha           : 2026-10-07
Cliente         : Tienda La Esperanza   (NIT 1234567-8)
Vendedor        : Carlos Méndez López
Condición       : Contado
------------------------------------------------------------------------
Código    Producto                    Cant.    P. unit.      Subtotal
------------------------------------------------------------------------
P001      Jabón líquido multiusos 1L      10       25.00        250.00
P003      Cloro 1 galón                    5       24.00        120.00
------------------------------------------------------------------------
Total de registros: 2
Subtotal : Q 370.00
IVA 12%  : Q 44.40
TOTAL    : Q 414.40
Saldo    : Q 0.00
================================================
```

Si se pide más cantidad de la que hay en existencia, el sistema lo rechaza y permite corregir:

Figura 9. Validación de existencia insuficiente.

```
Código de producto (Enter para terminar): P004
Cantidad (disponible 50): 100
  Existencia insuficiente. Solo hay 50 unidades disponibles.
```

Figura 10. Factura a crédito a 30 días: el saldo inicia en el total y vence en la fecha indicada.

```
==================== FACTURA ====================
No. correlativo : 2     Serie: A
Fecha           : 2026-10-07
Cliente         : Mini Super San José   (NIT 9988776-K)
Vendedor        : Carlos Méndez López
Condición       : Crédito  (plazo 30 días, vence 2026-11-06)
-------------------------------------------------------------------------
Código    Producto                    Cant.    P. unit.      Subtotal
-------------------------------------------------------------------------
P002      Desinfectante pino 1 galón      12       48.00        576.00
P001      Jabón líquido multiusos 1L      20       25.00        500.00
-------------------------------------------------------------------------
Total de registros: 2
Subtotal : Q 1076.00
IVA 12%  : Q 129.12
TOTAL    : Q 1205.12
Saldo    : Q 1205.12
================================================
```

Un Vendedor solo puede facturar a los clientes de su cartera:

Figura 11. Intento de facturar a un cliente de otro vendedor.

```
ID del cliente: 2
Ese cliente no pertenece a su cartera. No puede facturarle.
```

La opción **2. Consultar factura** permite volver a ver cualquier factura emitida.

4.5 Pagos

Seleccione **Pagos → Registrar pago**. El sistema lista las facturas con saldo pendiente; elija una e indique el monto. Se pueden registrar varios pagos parciales hasta cubrir el saldo, pero no se acepta un monto mayor al saldo pendiente.

Figura 12. Registro de un pago parcial.

```
--- Registrar pago ---
Facturas con saldo pendiente:
-------------------------------------------------------------------------------------
 Factura  Fecha       Cliente                Condición          Total         Saldo
-------------------------------------------------------------------------------------
       2  2026-10-07  Mini Super San José     Crédito          1205.12       1205.12
-------------------------------------------------------------------------------------
Total de registros: 1
Número de factura a abonar: 2
Saldo pendiente: Q 1205.12
Monto del pago (Q): 500
Pago registrado. Saldo restante de la factura 2: Q 705.12
```

Figura 13. Un monto mayor al saldo es rechazado.

```
Número de factura a abonar: 2
Saldo pendiente: Q 705.12
Monto del pago (Q): 5000
El monto (Q 5000.00) excede el saldo pendiente. Pago no registrado.
```

Figura 14. Consulta de los pagos de una factura (opción 2).

```
--- Pagos de una factura ---
Número de factura: 2
-----------------------------------------------
  Pago  Fecha              Monto         Saldo
-----------------------------------------------
     2  2026-10-07        500.00        705.12
     3  2026-10-07        400.00        305.12
-----------------------------------------------
Total de registros: 2
Saldo actual de la factura: Q 305.12
```

4.6 Reportes (Administrador)

Reporte	Qué muestra
Reporte de Clientes	Todos los clientes con sus datos de contacto y vendedor asignado
Reporte de Vendedores	Empleados de tipo Vendedor con sus datos y cantidad de clientes
Reporte de Productos Vendidos	Unidades y monto vendido por producto; opcionalmente por rango de fechas (AAAA-MM-DD). Enter en ambas fechas = sin límite
Listado de Ventas (Facturas del Día)	Facturas emitidas hoy con subtotal, IVA, total y gran total del día
Reporte de Pagos del Día	Pagos recibidos hoy y total cobrado
Reporte de Inventario Existente	Productos con costo, precio y existencia actual

Figura 15. Reporte de Clientes.

```
--- REPORTE DE CLIENTES ---
--------------------------------------------------------------------------------
  ID  Nombre                   NIT          Teléfono    Contacto            Vendedor
--------------------------------------------------------------------------------
   1  Tienda La Esperanza      1234567-8    2234-5678   María Gómez         V001
      Dirección: 6a Avenida 3-45, Zona 1  |  E-mail: laesperanza@gmail.com  |  Web: www.laesperanza.com
   2  Abarrotes El Chapín      7654321-0    2245-1122   Luis Pérez          V002
      Dirección: Calzada Roosevelt, Zona 7  |  E-mail: elchapin@hotmail.com
   3  Mini Super San José      9988776-K    2255-0099   Rosa Díaz           V001
      Dirección: Avenida Petapa, Zona 12  |  E-mail: minisuper@gmail.com
--------------------------------------------------------------------------------
Total de registros: 3
```

Figura 16. Reporte de Vendedores.

```
--- REPORTE DE VENDEDORES ---
-------------------------------------------------------------------------------------------
Código   DPI            Nombre                 Teléfono   E-mail                 Clientes
-------------------------------------------------------------------------------------------
V001     2456789010101  Carlos Méndez López    5512-3456  cmendez@chapina.com          2
      Dirección: Zona 5, Ciudad de Guatemala
V002     2987654320101  Ana Lucía Ruiz         5523-7788  aruiz@chapina.com            1
      Dirección: Zona 11, Guatemala
-------------------------------------------------------------------------------------------
Total de registros: 2
```

Figura 17. Reporte de Productos Vendidos.

```
--- REPORTE DE PRODUCTOS VENDIDOS ---
Desde (AAAA-MM-DD, Enter = sin límite):
Hasta (AAAA-MM-DD, Enter = sin límite):
------------------------------------------------------------------
Código    Producto                      Unidades   Monto vendido
------------------------------------------------------------------
P001      Jabón líquido multiusos 1L          36          900.00
P003      Cloro 1 galón                       15          360.00
P002      Desinfectante pino 1 galón          12          576.00
P004      Escoba plástica                      4          120.00
------------------------------------------------------------------
Total de registros: 4
```

Figura 18. Listado de Ventas (Facturas del Día).

```
--- LISTADO DE VENTAS (FACTURAS DEL DÍA) ---
-------------------------------------------------------------------------------------------------------------------
 Factura  Serie  Cliente                Vendedor               Condición     Subtotal        IVA        Total
-------------------------------------------------------------------------------------------------------------------
       1  A      Tienda La Esperanza    Carlos Méndez López    Contado         370.00      44.40       414.40
       2  A      Mini Super San José    Carlos Méndez López    Crédito        1076.00     129.12      1205.12
       3  A      Abarrotes El Chapín    Ana Lucía Ruiz         Crédito         360.00      43.20       403.20
       4  A      Abarrotes El Chapín    Ana Lucía Ruiz         Contado         150.00      18.00       168.00
-------------------------------------------------------------------------------------------------------------------
Total de registros: 4
TOTAL VENDIDO DEL DÍA: Q 2190.72
```

Figura 19. Reporte de Pagos del Día.

```
--- REPORTE DE PAGOS DEL DÍA ---
-------------------------------------------------------------------------------
  Pago   Factura  Cliente                  Condición          Monto         Saldo
-------------------------------------------------------------------------------
     1         1  Tienda La Esperanza      Contado           414.40          0.00
     2         2  Mini Super San José      Crédito           500.00        705.12
     3         2  Mini Super San José      Crédito           400.00        305.12
     4         4  Abarrotes El Chapín      Contado           168.00          0.00
-------------------------------------------------------------------------------
Total de registros: 4
TOTAL RECIBIDO DEL DÍA: Q 1482.40
```

Figura 20. Reporte de Inventario Existente.

```
--- REPORTE DE INVENTARIO EXISTENTE ---
-----------------------------------------------------------------------------
Código    Producto                         Costo      Precio  Existencia
-----------------------------------------------------------------------------
P001      Jabón líquido multiusos 1L        18.50       25.00          84
P002      Desinfectante pino 1 galón        32.00       48.00          48
P003      Cloro 1 galón                     15.00       24.00          65
P004      Escoba plástica                   20.00       30.00          46
-----------------------------------------------------------------------------
Total de registros: 4
```

5. Mensajes del sistema y cómo resolverlos

Mensaje	Causa y solución
Credenciales incorrectas (intento N de 3)	Usuario o clave erróneos. Verifique el código de empleado y use el DPI como clave. Al tercer fallo el sistema se cierra.
Ya existe un empleado con ese código / DPI	El dato debe ser único. Revise el listado de empleados antes de registrar.
Ya existe un cliente con ese NIT	El cliente ya está registrado. Búsquelo en Consultar clientes.
Ese código no corresponde a un Vendedor registrado	Solo un empleado de tipo Vendedor puede tener clientes. Elija un código de la lista mostrada.
Ese cliente no pertenece a su cartera	Un Vendedor solo factura a sus propios clientes. Elija un ID de la lista.
Existencia insuficiente. Solo hay N unidades	Pida una cantidad menor o solicite al Administrador el ingreso de mercadería.
El monto excede el saldo pendiente	Ingrese un monto menor o igual al saldo que muestra el sistema.
Esa factura no tiene saldo pendiente	La factura ya está pagada (las de contado nacen pagadas) o no pertenece al usuario.
Valor inválido. Ingrese un número…	Se escribió texto, un negativo o un valor fuera de rango. Vuelva a ingresarlo.

Figura 21. Intentos de acceso con credenciales incorrectas.

```
======================================================
  SISTEMA DE FACTURACIÓN - REPRESENTACIONES "CHAPINA"
======================================================
Usuario (admin o código de empleado): mal
Clave: 123
Credenciales incorrectas (intento 1 de 3).
Usuario (admin o código de empleado): admin
Clave: mala
Credenciales incorrectas (intento 2 de 3).
Usuario (admin o código de empleado): admin
Clave: admin123

Bienvenido(a), Administrador
```

PARTE 2: REVISIÓN DE CÓDIGO

6. Información general

Aspecto	Detalle
Lenguaje y estándar	C++17
Base de datos	SQLite 3, archivo único `chapina.db`, acceso mediante la API C (`sqlite3.h`)
Archivo fuente	`main.cpp`, 1,018 líneas (incluye comentarios y líneas en blanco)
Funciones	Más de 45 funciones, agrupadas en 9 secciones comentadas
Compilación	Sin advertencias con `-Wall -Wextra`
Archivos entregados	`main.cpp` y `chapina_demo.db` (base con datos de ejemplo)

7. Estructura del código

El programa está en un solo archivo, dividido en nueve secciones. Cada módulo funcional del manual corresponde a una sección:

Sección	Contenido	Funciones principales
1. Entrada / salida	Lectura validada de datos y tablas en pantalla	`leer`, `leerEntero`, `leerDecimal`, `confirmar`, `imprimirTabla`
2. Base de datos	Ejecución de SQL, creación de tablas, errores	`ejecutar`, `crearEsquema`, `traducirError`, `saldoFactura`
3. Empleados	Vendedores y Repartidores	`registrarEmpleado`, `modificarEmpleado`, `listarEmpleados`
4. Clientes	Cartera de clientes e historial	`registrarCliente`, `modificarCliente`, `historialCliente`
5. Productos	Catálogo e inventario	`registrarProducto`, `modificarProducto`, `reabastecerProducto`
6. Facturación	Facturas al contado y crédito	`generarFactura`, `mostrarFactura`, `consultarFactura`
7. Pagos	Pagos parciales y saldos	`registrarPago`, `consultarPagos`
8. Reportes	Los seis reportes requeridos	`reporteClientes`, `reporteVendedores`, `reporteProductosVendidos`, `reporteVentasDelDia`, `reportePagosDelDia`, `reporteInventario`
9. Menús y sesión	Navegación, login y roles	`correrMenu`, `iniciarSesion`, `main`

8. Modelo de datos

La base de datos tiene seis tablas, que corresponden a las entidades identificadas en el análisis de requerimientos. La generalización pedida para el personal se resolvió con una única tabla **Empleado** y una columna `tipo` restringida a los valores `Vendedor` y `Repartidor`.

Tabla	Llave primaria	Llaves foráneas	Restricciones
Empleado	`codigo`	—	`dpi` único; `tipo` ∈ {Vendedor, Repartidor}
Cliente	`id` (autoincremental)	`vendedor_codigo` → Empleado	`nit` único

Tabla	Llave primaria	Llaves foráneas	Restricciones
Producto	`codigo`	—	costo, precio y existencia ≥ 0
Factura	`correlativo` (autoincremental)	`cliente_id` → Cliente; `vendedor_codigo` → Empleado	`condicion` ∈ {Contado, Crédito}
DetalleFactura	`id`	`factura_correlativo` → Factura; `producto_codigo` → Producto	`cantidad` > 0
Pago	`correlativo` (autoincremental)	`factura_correlativo` → Factura	`monto` > 0; `saldo` ≥ 0

Relaciones: un Vendedor tiene muchos Clientes; un Cliente tiene muchas Facturas; una Factura tiene muchos renglones de detalle (cada uno de un Producto) y muchos Pagos. El saldo de una factura se calcula como el total menos la suma de sus pagos.

Figura 22. Fragmento de crearEsquema: tablas Empleado y Factura.

```
CREATE TABLE IF NOT EXISTS Empleado (
    codigo    TEXT PRIMARY KEY,
    dpi       TEXT NOT NULL UNIQUE,
    nombre    TEXT NOT NULL,
    telefono  TEXT,
    direccion TEXT,
    email     TEXT,
    tipo      TEXT NOT NULL CHECK (tipo IN ('Vendedor','Repartidor'))
);

CREATE TABLE IF NOT EXISTS Factura (
    correlativo       INTEGER PRIMARY KEY AUTOINCREMENT,
    serie             TEXT NOT NULL,
    fecha             TEXT NOT NULL,
    cliente_id        INTEGER NOT NULL REFERENCES Cliente(id),
    vendedor_codigo   TEXT NOT NULL REFERENCES Empleado(codigo),
    condicion         TEXT NOT NULL CHECK (condicion IN ('Contado','Crédito')),
    plazo             INTEGER NOT NULL DEFAULT 0,
    fecha_vencimiento TEXT NOT NULL,
    subtotal          REAL NOT NULL,
    iva               REAL NOT NULL,
    total             REAL NOT NULL
);
```

9. Decisiones de diseño y buenas prácticas

- **Consultas parametrizadas.** Todo dato escrito por el usuario viaja como parámetro (?) de una sentencia preparada, nunca concatenado en el texto SQL. Esto evita la inyección de SQL.

Figura 23. Función ejecutar: sentencias preparadas con parámetros.

```
static bool ejecutar(const string& sql, const Fila& params = {}, Tabla* salida = nullptr) {
    sqlite3_stmt* st = nullptr;
    if (sqlite3_prepare_v2(db, sql.c_str(), -1, &st, nullptr) != SQLITE_OK) {
        cout << "  Error interno de base de datos: " << sqlite3_errmsg(db) << "\n";
        return false;
    }
    for (size_t i = 0; i < params.size(); i++)
        sqlite3_bind_text(st, (int)i + 1, params[i].c_str(), -1, SQLITE_TRANSIENT);

    int cols = sqlite3_column_count(st);
    int rc;
    while ((rc = sqlite3_step(st)) == SQLITE_ROW) {
        if (salida) {
            Fila f;
            for (int c = 0; c < cols; c++) {
                const unsigned char* t = sqlite3_column_text(st, c);
                f.push_back(t ? reinterpret_cast<const char*>(t) : "");
            }
            salida->push_back(f);
        }
    }
    bool ok = (rc == SQLITE_DONE);
```

- **Transacciones.** Emitir una factura toca varias tablas (factura, detalle, inventario y, si es contado, el pago). Todo se guarda dentro de una transacción: si algún paso falla, se revierte y no queda información a medias.

Figura 24. Fragmento de generarFactura: confirmación o reversión de la transacción.

```
    // Todo se guarda en una transacción: o se guarda completo o no se guarda nada.
    ejecutar("BEGIN");
    bool ok = ejecutar("INSERT INTO
Factura(serie,fecha,cliente_id,vendedor_codigo,condicion,plazo,fecha_vencimiento,subtotal,iva,total) "
                       "VALUES('A', date('now','localtime'), ?, ?, ?, ?, date('now','localtime','+' || ? || '
days'), ?, ?, ?)",
                       {to_string(cid), c[0][2], cond == 1 ? "Contado" : "Crédito", to_string(plazo), to_string(plazo),
                        num2(subtotal), num2(iva), num2(total)});
    string correlativo = to_string((long long)sqlite3_last_insert_rowid(db));

    for (size_t i = 0; ok && i < lineas.size(); i++) {
        const Linea& l = lineas[i];
        ok = ejecutar("INSERT INTO DetalleFactura(factura_correlativo,producto_codigo,cantidad,precio_unitario,subtotal)
VALUES(?,?,?,?,?)",
                      {correlativo, l.codigo, to_string(l.cantidad), num2(l.precio), num2(redondear(l.cantidad * l.precio))});
        if (ok) {
            ok = ejecutar("UPDATE Producto SET existencia = existencia - ? WHERE codigo = ? AND existencia >= ?",
                          {to_string(l.cantidad), l.codigo, to_string(l.cantidad)});
            if (ok && sqlite3_changes(db) != 1) ok = false;
        }
    }
    if (ok && cond == 1)  // Contado: un solo pago por el total
        ok = ejecutar("INSERT INTO Pago(factura_correlativo,fecha,monto,saldo) VALUES(?, date('now','localtime'), ?, 0)",
                      {correlativo, num2(total)});

    if (ok) {
        ejecutar("COMMIT");
        cout << "\nFactura generada correctamente.\n";
        mostrarFactura(correlativo);
    } else {
        ejecutar("ROLLBACK");
        cout << "No se pudo generar la factura. No se realizó ningún cambio.\n";
    }
```

- **Integridad en la propia base de datos.** Las reglas de negocio críticas (códigos y NIT únicos, valores no negativos, tipos permitidos, llaves foráneas con `PRAGMA foreign_keys = ON`) están declaradas en el esquema, de modo que se cumplen aunque falle una validación del programa.
- **Validación de entradas.** `leerEntero` y `leerDecimal` repiten la pregunta hasta recibir un valor válido; los errores de la base de datos se traducen a mensajes comprensibles (`traducirError`).
- **Control de acceso por rol.** El Vendedor solo ve y factura a los clientes de su cartera (filtro por `vendedor_codigo` en cada consulta); las opciones de Empleados, Productos y Reportes solo aparecen para el Administrador.
- **Cálculo monetario.** El IVA se define en una constante (`TASA_IVA = 0.12`) y cada importe se redondea a dos decimales antes de guardarse.

10. Trazabilidad con los requerimientos

Cada requerimiento del catálogo (REQ-0001 a REQ-0010) tiene su implementación en el código:

Requerimiento	Implementación
REQ-0001 Empleados (Vendedor/Repartidor) y cartera	Tabla `Empleado` con `tipo`; `registrarEmpleado`, `modificarEmpleado`, `listarEmpleados`; `Cliente.vendedor_codigo`

Requerimiento	Implementación
REQ-0002 Clientes e historial	`registrarCliente`, `modificarCliente`, `consultarClientes`, `historialCliente`
REQ-0003 Productos e inventario	`registrarProducto`, `reabastecerProducto`; descuento de existencia y rechazo por existencia insuficiente en `generarFactura`
REQ-0004 Factura, detalle e IVA	`generarFactura` y `mostrarFactura`; tablas `Factura` y `DetalleFactura`
REQ-0005 Pagos y saldo	Pago único automático en facturas de contado (`generarFactura`); `registrarPago` para pagos parciales; `saldoFactura`
REQ-0006 Seis reportes	`reporteClientes`, `reporteVendedores`, `reporteProductosVendidos`, `reporteVentasDelDia`, `reportePagosDelDia`, `reporteInventario`
REQ-0007 Interfaz sencilla	Menús numerados (`correrMenu`), mensajes claros y confirmación antes de guardar una factura
REQ-0008 Rendimiento y disponibilidad	Base de datos local SQLite, sin dependencia de red ni de un servidor; reportes resueltos con consultas agregadas
REQ-0009 Autenticación y validación	`iniciarSesion` con perfiles; restricciones `CHECK`/`UNIQUE`; `leerEntero` y `leerDecimal`
REQ-0010 Crecimiento y equipos actuales	Un solo ejecutable y un solo archivo `.db`; código portable a Windows, Linux y macOS

11. Pruebas realizadas

Durante el desarrollo se ejecutó el sistema con cinco sesiones de prueba (Administrador, dos Vendedores, reportes y casos de error). Los resultados principales:

Prueba	Resultado
Compilación con `-Wall -Wextra`	Sin errores ni advertencias
Factura al contado con IVA	Subtotal Q 370.00 + IVA Q 44.40 = Q 414.40; pago único automático, saldo Q 0.00
Factura a crédito (30 días)	Total Q 1,205.12; vencimiento calculado: 06/11/2026
Descuento de inventario	Jabón líquido: 120 → 84 unidades tras vender 36 en tres facturas
Existencia insuficiente	Pedido de 100 unidades con 50 disponibles: rechazado
Pagos parciales	Q 1,205.12 → Q 705.12 → Q 305.12; un pago de Q 5,000.00 fue rechazado
Cartera del Vendedor	Un Vendedor no pudo facturar a un cliente de otro vendedor
Datos duplicados e inválidos	Código de empleado y NIT repetidos rechazados; textos y negativos rechazados
Integridad de la base de datos	`PRAGMA integrity_check` = ok; `PRAGMA foreign_key_check` sin violaciones; saldos coherentes con los pagos

El detalle de los casos de prueba se presenta en el Plan de Pruebas (Entregable 8).

12. Limitaciones y mejoras futuras

- La clave del Administrador está fija en el código para fines de demostración; en producción debería almacenarse cifrada y poder cambiarse.
- El sistema no contempla anulación de facturas, devoluciones ni notas de crédito.
- La facturación electrónica ante la SAT (FEL) y la integración con sistemas contables quedaron fuera del alcance, como se documentó en la propuesta comercial.

- La interfaz es de consola; una interfaz gráfica o web sería una evolución natural.
- Los reportes se muestran en pantalla; podrían exportarse a archivo o PDF.

PARTE 1: PLANIFICACIÓN DE LAS PRUEBAS

1. Introducción

Este documento es el Plan de Pruebas del Sistema de Facturación de Representaciones "Chapina", C.A., una aplicación de consola desarrollada en C++ con base de datos SQLite. Define qué se va a probar, cómo, con qué datos y bajo qué criterios se considera que el sistema está listo para entregarse.

El plan se apoya en los documentos anteriores del proyecto: los 10 requerimientos del catálogo (REQ-0001 a REQ-0010), los casos de uso del Entregable 4 y el manual de usuario del Entregable 7. Cada caso de prueba queda enlazado a un requerimiento, de modo que se pueda comprobar que nada de lo prometido quedó sin verificar.

2. Objetivos

2.1 Objetivo general

Verificar que el sistema cumple los requerimientos funcionales y no funcionales acordados, y que opera de forma correcta y confiable en el trabajo diario de la empresa.

2.2 Objetivos específicos

- Comprobar que cada módulo (empleados, clientes, productos, facturación, pagos y reportes) hace lo que se espera con datos válidos.
- Comprobar que el sistema rechaza datos inválidos con mensajes claros y sin dañar la base de datos.
- Verificar que los cálculos monetarios (subtotal, IVA del 12 %, total y saldo) son exactos.
- Verificar que el inventario se descuenta correctamente al facturar y que nunca queda en negativo.
- Confirmar que el control de acceso por rol (Administrador y Vendedor) funciona.
- Confirmar que la integridad de la base de datos se mantiene después de todas las pruebas.
- Revisar la calidad del código fuente: que compile sin advertencias, que no tenga errores de memoria, que sus consultas SQL sean seguras y que las pruebas recorran la mayor parte de sus líneas.

3. Alcance

3.1 Qué se prueba

Módulo	Funciones incluidas	Requerimiento
Acceso y seguridad	Inicio de sesión, intentos fallidos, permisos por rol	REQ-0009
Empleados	Registrar, modificar, consultar; tipo Vendedor o Repartidor	REQ-0001
Clientes	Registrar, modificar, consultar, historial; asignación a vendedor	REQ-0002
Productos e inventario	Registrar, modificar, ingreso de mercadería, consulta	REQ-0003
Facturación	Factura de contado y crédito, detalle, IVA, descuento de inventario	REQ-0004
Pagos	Pago completo y parcial, saldo, rechazo de abonos mayores al saldo	REQ-0005
Reportes	Los seis reportes solicitados por la gerencia	REQ-0006
Interfaz y rendimiento	Menús claros, mensajes de error, tiempo de respuesta	REQ-0007, REQ-0008
Integridad de datos	Llaves foráneas, restricciones, transacciones	REQ-0009, REQ-0010

Módulo	Funciones incluidas	Requerimiento
Calidad del código	Advertencias del compilador, errores de memoria, cobertura de líneas, seguridad de las consultas SQL	REQ-0004, REQ-0009

3.2 Qué no se prueba

- Pruebas de carga con miles de usuarios simultáneos: el sistema es de un solo usuario por equipo (ver Entregable 5).
- Impresión física de facturas y conexión con la SAT (certificación FEL), que quedan fuera del alcance del proyecto.
- Pruebas sobre sistemas operativos distintos de los indicados en el entorno.

4. Estrategia de pruebas

Se combinan cuatro enfoques, de lo más pequeño a lo más completo:

Nivel	Qué busca	Cómo se hace
Pruebas de validación (caja negra)	Que cada función responda bien a entradas válidas e inválidas	Se ejecuta el programa, se digitan los datos del caso y se compara lo que aparece en pantalla con el resultado esperado.
Pruebas de integración	Que los módulos trabajen juntos: una factura debe afectar inventario, saldo, pagos y reportes	Se sigue un flujo completo: registrar, facturar, pagar y consultar reportes.
Pruebas sobre el código (caja blanca)	Que el código fuente sea correcto y seguro por dentro, no solo por fuera	Se compila con análisis de errores de memoria (AddressSanitizer), de comportamiento indefinido (UBSan) y cobertura (gcov); se prueban ramas específicas: transacciones, filtros por rol, inyección SQL y cierre inesperado.
Pruebas de base de datos	Que los datos queden bien guardados y consistentes	Se revisan las restricciones (PRIMARY KEY, UNIQUE, CHECK, FOREIGN KEY) y se ejecutan `PRAGMA integrity_check` y `PRAGMA foreign_key_check` al final.

En cada módulo se prueban tres tipos de casos: **camino normal** (datos correctos), **casos límite** (valores en el borde, como cantidad igual a la existencia o monto igual al saldo) y **casos de error** (datos vacíos, duplicados, negativos o sin permiso).

4.1 Técnicas de diseño de casos

- **Partición de equivalencia:** se prueba un valor representativo de cada grupo (válido, vacío, negativo, texto en lugar de número).
- **Valores límite:** cantidad = existencia, cantidad = existencia + 1, abono = saldo, abono = saldo + 0.01.
- **Flujo de negocio:** se prueban de principio a fin los escenarios reales de la empresa (venta al contado, venta a crédito con abonos).

5. Entorno de pruebas

Elemento	Detalle
Sistema operativo	Linux (Ubuntu 24.04). El código incluye soporte para Windows (UTF-8 en consola).
Compilador	g++ 13.3.0 con `-std=c++17 -Wall -Wextra`
Base de datos	SQLite 3.45.1 (archivo local `chapina.db`)
Comando de compilación	`g++ -std=c++17 -Wall -Wextra -o chapina main.cpp -lsqlite3`
Ejecución	Interactiva por consola. Las entradas se guardan en archivos de texto para poder repetir cada prueba con exactamente los mismos datos.
Probador	Werner Cifuentes (proyecto individual)

6. Datos de prueba

Antes de empezar se parte de una base de datos vacía. Los datos de ejemplo que se registran durante las pruebas son los siguientes:

Tipo	Datos
Administrador	Usuario `admin`, clave `admin123`
Empleados	V001 Carlos Méndez López (Vendedor, DPI 2456789010101); V002 Ana Lucía Ruiz (Vendedor, DPI 2987654320101); R001 Pedro Sicán (Repartidor, DPI 3011223340101)
Clientes	Tienda La Esperanza (NIT 1234567-8, vendedor V001); Abarrotes El Chapín (NIT 7654321-0, V002); Mini Super San José (NIT 9988776-K, V001)
Productos	P001 Jabón líquido multiusos 1L (Q25.00, existencia 120); P002 Desinfectante pino 1 galón (Q48.00, 60); P003 Cloro 1 galón (Q24.00, 80); P004 Escoba plástica (Q30.00, 35)

Se eligieron datos realistas para la empresa y con montos fáciles de verificar a mano. Por ejemplo, 10 unidades de P001 a Q25.00 dan Q250.00 de subtotal; el IVA es Q30.00 y el total Q280.00.

7. Criterios de las pruebas

7.1 Criterios de entrada

- El programa compila sin errores ni advertencias.
- La base de datos se crea sola al primer arranque, con todas sus tablas.
- El manual de usuario (Entregable 7) está terminado, porque describe el comportamiento esperado.

7.2 Criterios de aprobación de un caso

Un caso se marca como **Aprobado** cuando el resultado obtenido es igual al esperado. Si hay diferencia, se marca **Fallido**, se registra como defecto y se corrige antes de repetir la prueba.

7.3 Severidad de los defectos

Severidad	Significado	Ejemplo
Crítica	Pérdida o corrupción de datos, o el sistema se cierra	Una factura descuenta inventario pero no se guarda

Severidad	Significado	Ejemplo
Alta	Cálculo o regla de negocio incorrecta	El IVA no es el 12 %
Media	Una función no responde como se espera, con solución alterna	Un reporte no muestra un campo
Baja	Detalle visual o de redacción	Una columna desalineada

7.4 Criterios de salida

El sistema se considera listo para entregarse cuando:

- El 100 % de los casos de prioridad Alta está aprobado.
- No quedan defectos de severidad Crítica o Alta abiertos.
- Los seis reportes cuadran con los datos registrados (ventas, pagos e inventario).
- `PRAGMA integrity_check` devuelve `ok` y `PRAGMA foreign_key_check` no devuelve filas.

8. Formato de los casos de prueba

Cada caso se identifica con el código **CP-MÓDULO-NN**, por ejemplo CP-FAC-03 es el tercer caso del módulo de facturación. Los códigos de módulo son: ACC (acceso), EMP (empleados), CLI (clientes), PRO (productos), FAC (facturación), PAG (pagos), REP (reportes), INT (integridad), REN (rendimiento) y COD (código). Cada caso lleva los siguientes campos:

Campo	Descripción
ID	Código único del caso
Requerimiento	REQ al que da cobertura
Descripción	Qué se quiere comprobar
Datos de entrada	Valores que se digitan
Resultado esperado	Lo que debe mostrar el sistema
Resultado obtenido	Lo que mostró realmente
Prioridad	Alta, Media o Baja
Estado	Aprobado o Fallido

9. Cronograma de pruebas

Actividad	Fecha	Entrega
Planificación de las pruebas (Parte 1)	13/10/2026	Este documento
Diseño y ejecución de casos por módulo (Parte 2)	13/10/2026	Casos y resultados
Resultados, defectos y conclusiones (Parte 3)	13/10/2026	Informe final
Entrega del Plan de Pruebas	14/10/2026	Entregable 8

PARTE 2: CASOS DE PRUEBA Y EJECUCIÓN

10. Cómo se ejecutaron las pruebas

Los casos se ejecutaron sobre el programa ya compilado, sin modificar su código. Para que los resultados sean repetibles se siguió este procedimiento en cada caso:

- Se parte de una copia de una base de datos conocida: **vacía**, **base inicial** (3 empleados, 2 clientes y 4 productos) o **flujo completo** (la base inicial más 4 facturas y 4 pagos).
- Los datos del caso se envían al programa tal como los digitaría el usuario, por la entrada estándar. El programa repite en pantalla cada dato recibido, así que la salida muestra exactamente lo que vería el usuario.
- Se compara la pantalla con el resultado esperado (mensajes, montos y listados).
- Cuando el caso modifica datos, también se consulta la base de datos para confirmar lo guardado (por ejemplo, la existencia de un producto o el saldo de un pago).

Un caso solo se marca **Aprobado** si coinciden la pantalla y la base de datos. Para revisar también el código, los casos se ejecutaron con una versión del programa compilada con instrumentación: detección de errores de memoria (AddressSanitizer), de comportamiento indefinido (UBSan) y medición de cobertura (gcov). En total se ejecutaron **90 casos** de prueba, agrupados por módulo en las siguientes tablas.

11. Casos de prueba por módulo

11.1 Acceso y seguridad (REQ-0009)

ID	Req. / Prior.	Descripción y datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-ACC-01	REQ-0009, Alta	**Inicio de sesión del Administrador.** Datos: admin / admin123	Mensaje de bienvenida y menú con 6 opciones	Bienvenido(a), Administrador	**Aprobado**
CP-ACC-02	REQ-0009, Alta	**Inicio de sesión de un Vendedor (código y DPI).** Datos: V001 / 2456789010101	Bienvenida con rol Vendedor y menú reducido (Clientes, Facturación, Pagos)	Bienvenido(a), Carlos Méndez López (Vendedor)	**Aprobado**
CP-ACC-03	REQ-0009, Alta	**Contraseña incorrecta y luego correcta.** Datos: mal/123, admin/mala, admin/admin123	Avisa intento 1 de 3 y 2 de 3; al tercero entra	Credenciales incorrectas (intento 1 de 3). / Credenciales incorrectas (intento 2 de 3). / Bienvenido(a), Administrador	**Aprobado**
CP-ACC-04	REQ-0009, Alta	**Tres intentos fallidos cierran el sistema.** Datos: 3 pares de credenciales incorrectas	Mensaje de demasiados intentos y cierre	Demasiados intentos fallidos. El sistema se cerrará.	**Aprobado**
CP-ACC-05	REQ-0009, Alta	**Un Repartidor no puede iniciar sesión.** Datos: R001 / 3011223340101	Credenciales incorrectas	Credenciales incorrectas (intento 1 de 3).	**Aprobado**
CP-ACC-06	REQ-0009, Media	**Vendedor con DPI incorrecto.** Datos: V001 / 0000000000000	Credenciales incorrectas	Credenciales incorrectas (intento 1 de 3).	**Aprobado**

11.2 Empleados (REQ-0001)

ID	Req. / Prior.	Descripción y datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-EMP-01	REQ-0001, Alta	**Registrar un Vendedor.** Datos: V003, DPI 4444444440101, Luis Barrios, tipo 1	Empleado registrado; tipo Vendedor en la BD	Empleado registrado correctamente. [Valor en BD: Vendedor]	**Aprobado**

ID	Req. / Prior.	Descripción y datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-EMP-02	REQ-0001, Alta	**Registrar un Repartidor (generalización por tipo).** Datos: R002, DPI 5555555550101, Mario Tzoc, tipo 2	Empleado registrado; tipo Repartidor en la BD	Empleado registrado correctamente. [Valor en BD: Repartidor]	**Aprobado**
CP-EMP-03	REQ-0001, Alta	**Código de empleado duplicado.** Datos: V001 (ya existe)	Rechaza con mensaje claro	No se pudo completar la operación: Ya existe un empleado con ese código. [Valor en BD: 3]	**Aprobado**
CP-EMP-04	REQ-0001, Alta	**DPI duplicado.** Datos: V009 con DPI 2456789010101 (de V001)	Rechaza; no se crea el empleado	No se pudo completar la operación: Ya existe un empleado con ese DPI. [Valor en BD: 0]	**Aprobado**
CP-EMP-05	REQ-0001, Media	**Dato obligatorio vacío (código).** Datos: Enter en blanco, luego V010	Avisa que el dato es obligatorio y vuelve a pedirlo	Este dato es obligatorio. / Empleado registrado correctamente.	**Aprobado**
CP-EMP-06	REQ-0001, Media	**Tipo de empleado fuera de rango.** Datos: Tipo 5, luego 1	Rechaza el 5 y vuelve a pedir	Valor inválido. Ingrese un número entero entre 1 y 2. / Empleado registrado correctamente.	**Aprobado**
CP-EMP-07	REQ-0001, Media	**Modificar el teléfono de un empleado.** Datos: V001, teléfono nuevo 5599-0000	Dato actualizado en la BD	Empleado actualizado correctamente. [Valor en BD: 5599-0000]	**Aprobado**
CP-EMP-08	REQ-0001, Baja	**Modificar un empleado que no existe.** Datos: Código ZZZ	Mensaje de no encontrado	No existe un empleado con ese código.	**Aprobado**
CP-EMP-09	REQ-0001, Media	**Consultar empleados.** Datos: Opción Consultar	Lista con 3 empleados y su tipo	Total de registros: 3	**Aprobado**

11.3 Clientes (REQ-0002)

ID	Req. / Prior.	Descripción y datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-CLI-01	REQ-0002, Alta	**Registrar cliente asignado a un vendedor.** Datos: Distribuidora Santa Rosa, NIT 4455667-1, vendedor V001	Cliente registrado	Cliente registrado correctamente. [Valor en BD: V001]	**Aprobado**
CP-CLI-02	REQ-0002, Alta	**NIT duplicado.** Datos: NIT 1234567-8 (ya existe)	Rechaza con mensaje claro	No se pudo completar la operación: Ya existe un cliente con ese NIT.	**Aprobado**
CP-CLI-03	REQ-0002, Alta	**Asignar cliente a un Repartidor.** Datos: Vendedor asignado: R001	Rechaza: solo vendedores	Ese código no corresponde a un Vendedor registrado. [Valor en BD: 0]	**Aprobado**
CP-CLI-04	REQ-0002, Media	**Nombre del cliente vacío.** Datos: Enter, luego un nombre	Pide el dato obligatorio	Este dato es obligatorio. / Cliente registrado correctamente.	**Aprobado**
CP-CLI-05	REQ-0002, Media	**Modificar teléfono de un cliente.** Datos: Cliente 1, teléfono 2299-0000	Dato actualizado	Cliente actualizado correctamente. [Valor en BD: 2299-0000]	**Aprobado**
CP-CLI-06	REQ-0002, Alta	**El vendedor solo ve su propia cartera.** Datos: V001 consulta clientes	Ve a Tienda La Esperanza, no ve a Abarrotes El Chapín (de V002)	1 Tienda La Esperanza 1234567-8 2234-5678 María Gómez V001 / Total de registros: 1	**Aprobado**

ID	Req. / Prior.	Descripción y datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-CLI-07	REQ-0002, Media	**Cliente nuevo registrado por un vendedor.** Datos: V001 registra Cliente Vend	Se asigna automáticamente a V001	Vendedor asignado: V001 (usted) / Cliente registrado correctamente. [Valor en BD: V001]	**Aprobado**
CP-CLI-08	REQ-0002, Alta	**Historial de un cliente con facturas y pagos.** Datos: Cliente 3 (Mini Super San José)	Muestra sus facturas y el saldo	3 Mini Super San José 9988776-K 2255-0099 Rosa Díaz V001 / Cliente: Mini Super San José NIT: 9988776-K / Total de registros: 3	**Aprobado**

11.4 Productos e inventario (REQ-0003)

ID	Req. / Prior.	Descripción y datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-PRO-01	REQ-0003, Alta	**Registrar producto válido.** Datos: P005 Trapeador, costo 20, precio 35, existencia 40	Producto registrado	Producto registrado correctamente. [Valor en BD: 40]	**Aprobado**
CP-PRO-02	REQ-0003, Alta	**Código de producto duplicado.** Datos: P001 (ya existe)	Rechaza con mensaje claro	No se pudo completar la operación: Ya existe un producto con ese código. [Valor en BD: Jabón líquido multiusos 1L]	**Aprobado**
CP-PRO-03	REQ-0003, Media	**Costo con texto en lugar de número.** Datos: Costo: abc, luego 10	Rechaza el texto y vuelve a pedir	Valor inválido. Ingrese un número mayor o igual a 0. / Producto registrado correctamente.	**Aprobado**
CP-PRO-04	REQ-0003, Media	**Existencia negativa.** Datos: Existencia: -3, luego 5	Rechaza el valor negativo	Valor inválido. Ingrese un número entero mayor o igual a 0. / Producto registrado correctamente.	**Aprobado**
CP-PRO-05	REQ-0003, Baja	**Precio de venta menor al costo.** Datos: Costo 10, precio 8	Muestra aviso pero permite guardar	Aviso: el precio de venta es menor al costo. / Producto registrado correctamente.	**Aprobado**
CP-PRO-06	REQ-0003, Media	**Modificar el precio de un producto.** Datos: P001, precio nuevo Q26.00	Precio actualizado en la BD	Producto actualizado correctamente. [Valor en BD: 26.0]	**Aprobado**
CP-PRO-07	REQ-0003, Alta	**Ingreso de mercadería.** Datos: P004, cantidad 15 (existencia inicial 50)	Existencia sube a 65	Existencia actualizada: Escoba plástica ahora tiene 65 unidades. [Valor en BD: 65]	**Aprobado**
CP-PRO-08	REQ-0003, Media	**Ingreso de mercadería con cantidad 0.** Datos: Cantidad 0, luego 5	Rechaza el 0	Valor inválido. Ingrese un número entero mayor o igual a 1. / Existencia actualizada: Escoba plástica ahora tiene 55 unidades. [Valor en BD: 55]	**Aprobado**
CP-PRO-09	REQ-0003, Baja	**Ingreso a un producto que no existe.** Datos: Código P999	Mensaje de no encontrado	No existe un producto con ese código.	**Aprobado**

11.5 Facturación (REQ-0004)

ID	Req. / Prior.	Descripción y datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-FAC-01	REQ-0004, Alta	**Factura al contado con IVA.** Datos: Cliente 1, contado, 10 x P001 (Q25) + 2 x P004 (Q30)	Subtotal 310.00, IVA 37.20, total 347.20; inventario baja; pago automático con saldo 0	Resumen: Subtotal Q 310.00 | IVA Q 37.20 | TOTAL Q 347.20 [Valor en BD: 110, 48, 0.0, 347.2]	**Aprobado**
CP-FAC-02	REQ-0004, Alta	**Factura a crédito con plazo.** Datos: Cliente 2, crédito 30 días, 12 x P002 (Q48)	Subtotal 576.00, IVA 69.12, total 645.12; sin pago; vence en 30 días	Resumen: Subtotal Q 576.00 | IVA Q 69.12 | TOTAL Q 645.12 [Valor en BD: 0, 30, 30.0]	**Aprobado**
CP-FAC-03	REQ-0004, Alta	**Cantidad mayor a la existencia (límite +1).** Datos: P003 existencia 80, cantidad 81	Rechaza la línea; sin productos la factura se cancela	Existencia insuficiente. Solo hay 80 unidades disponibles. / No se agregó ningún producto. Factura cancelada. [Valor en BD: 80, 0]	**Aprobado**
CP-FAC-04	REQ-0004, Alta	**Cantidad igual a la existencia (límite).** Datos: P003 existencia 80, cantidad 80	Acepta; existencia queda en 0; total 2,150.40	Resumen: Subtotal Q 1920.00 | IVA Q 230.40 | TOTAL Q 2150.40 [Valor en BD: 0]	**Aprobado**
CP-FAC-05	REQ-0004, Media	**Producto inexistente en la factura.** Datos: Código XXX	Avisa y no agrega la línea	Producto no encontrado. / No se agregó ningún producto. Factura cancelada.	**Aprobado**
CP-FAC-06	REQ-0004, Media	**Cliente inexistente.** Datos: ID 99	Mensaje de cliente no encontrado	Cliente no encontrado.	**Aprobado**
CP-FAC-07	REQ-0004, Alta	**Cancelar la factura al confirmar.** Datos: Responde N en la confirmación	No se guarda nada y el inventario no cambia	Factura cancelada. [Valor en BD: 0, 120]	**Aprobado**
CP-FAC-08	REQ-0004, Alta	**Vendedor factura a cliente de otro vendedor.** Datos: V001 intenta facturar al cliente 2 (de V002)	Rechaza: no es de su cartera	Ese cliente no pertenece a su cartera. No puede facturarle. [Valor en BD: 0]	**Aprobado**
CP-FAC-09	REQ-0004, Media	**Condición de pago inválida.** Datos: Condición 3, luego 1	Rechaza el 3	Valor inválido. Ingrese un número entero entre 1 y 2.	**Aprobado**
CP-FAC-10	REQ-0004, Media	**Plazo de crédito fuera de rango.** Datos: Plazo 400, luego 30	Rechaza el 400	Valor inválido. Ingrese un número entero entre 1 y 365.	**Aprobado**
CP-FAC-11	REQ-0004, Alta	**Mismo producto en dos líneas (acumulado).** Datos: P001: 100 y luego 21 (existencia 120)	La segunda línea ve solo 20 disponibles y rechaza 21	Cantidad (disponible 20): 21 / Existencia insuficiente. Solo hay 20 unidades disponibles. [Valor en BD: 20]	**Aprobado**
CP-FAC-12	REQ-0004, Alta	**Redondeo del IVA a 2 decimales.** Datos: 3 x producto a Q33.33 (subtotal 99.99)	IVA 12.00 (11.9988 redondeado), total 111.99	Resumen: Subtotal Q 99.99 | IVA Q 12.00 | TOTAL Q 111.99	**Aprobado**
CP-FAC-13	REQ-0004, Media	**Consultar el detalle de una factura.** Datos: Factura 1 (contado)	Muestra encabezado, líneas, IVA y total	TOTAL : Q 414.40	**Aprobado**
CP-FAC-14	REQ-0004, Media	**Los correlativos son consecutivos.** Datos: 4 facturas del flujo completo	Correlativos 1,2,3,4 sin saltos	Valor en BD: 1,2,3,4	**Aprobado**

11.6 Pagos (REQ-0005)

ID	Req. / Prior.	Descripción y datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-PAG-01	REQ-0005, Alta	**Pago parcial de una factura a crédito.** Datos: Factura 1 (Q645.12), abono Q300	Saldo restante Q345.12	Pago registrado. Saldo restante de la factura 1: Q 345.12 [Valor en BD: 345.12]	**Aprobado**
CP-PAG-02	REQ-0005, Alta	**Abono mayor al saldo.** Datos: Abono Q700 con saldo Q645.12	Rechaza; no se registra el pago	El monto (Q 700.00) excede el saldo pendiente. Pago no registrado. [Valor en BD: 0]	**Aprobado**
CP-PAG-03	REQ-0005, Alta	**Abono igual al saldo (límite).** Datos: Abono Q645.12	Saldo queda en 0.00	Pago registrado. Saldo restante de la factura 1: Q 0.00 [Valor en BD: 0.0]	**Aprobado**
CP-PAG-04	REQ-0005, Alta	**Abono de un centavo sobre el saldo (límite +0.01).** Datos: Abono Q645.13	Rechaza	El monto (Q 645.13) excede el saldo pendiente. Pago no registrado. [Valor en BD: 0]	**Aprobado**
CP-PAG-05	REQ-0005, Alta	**Dos pagos parciales seguidos.** Datos: Q300 y luego Q200	Saldos 345.12 y 145.12	Pago registrado. Saldo restante de la factura 1: Q 345.12 / Pago registrado. Saldo restante de la factura 1: Q 145.12 [Valor en BD: 345.12,145.12]	**Aprobado**
CP-PAG-06	REQ-0005, Media	**Monto cero o negativo.** Datos: Monto 0, luego 100	Rechaza el 0	Valor inválido. Ingrese un número mayor o igual a 0.01. / Pago registrado. Saldo restante de la factura 1: Q 545.12	**Aprobado**
CP-PAG-07	REQ-0005, Media	**Factura sin saldo pendiente.** Datos: Factura 99	Mensaje de que no está disponible	Esa factura no tiene saldo pendiente o no está disponible para usted.	**Aprobado**
CP-PAG-08	REQ-0005, Media	**Sin facturas pendientes.** Datos: Base sin facturas	Informa que no hay saldos pendientes	No hay facturas con saldo pendiente.	**Aprobado**
CP-PAG-09	REQ-0005, Alta	**Factura al contado nace pagada.** Datos: Factura contado Q347.20	Pago automático por el total y saldo 0	Condición (1=Contado, 2=Crédito): 1 / Condición : Contado [Valor en BD: 347.2, 0.0]	**Aprobado**
CP-PAG-10	REQ-0005, Media	**Consultar pagos de una factura.** Datos: Factura 2 del flujo (2 pagos)	Lista los 2 pagos y el saldo actual	Saldo actual de la factura: Q 305.12 / Total de registros: 2	**Aprobado**
CP-PAG-11	REQ-0005, Alta	**Vendedor no ve facturas de otro vendedor al pagar.** Datos: V001 con factura crédito de V002	No aparece la factura de V002	No hay facturas con saldo pendiente.	**Aprobado**

11.7 Reportes (REQ-0006)

ID	Req. / Prior.	Descripción y datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-REP-01	REQ-0006, Alta	**Reporte de Clientes.** Datos: Base del flujo completo	3 clientes con vendedor asignado	Total de registros: 3	**Aprobado**
CP-REP-02	REQ-0006, Alta	**Reporte de Vendedores.** Datos: Base del flujo completo	Solo vendedores (2) con cantidad de clientes; sin Repartidor	Total de registros: 2	**Aprobado**

ID	Req. / Prior.	Descripción y datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-REP-03	REQ-0006, Alta	**Reporte de Productos Vendidos.** Datos: Sin rango de fechas	P001 36 u. (900.00), P003 15 (360.00), P002 12 (576.00), P004 4 (120.00)	P001 Jabón líquido multiusos 1L 36 900.00 / P003 Cloro 1 galón 15 360.00 / P002 Desinfectante pino 1 galón 12 …	**Aprobado**
CP-REP-04	REQ-0006, Alta	**Ventas del Día.** Datos: Fecha de hoy	4 facturas; total vendido Q 2190.72	TOTAL VENDIDO DEL DÍA: Q 2190.72	**Aprobado**
CP-REP-05	REQ-0006, Alta	**Pagos del Día.** Datos: Fecha de hoy	4 pagos; total recibido Q 1482.40	TOTAL RECIBIDO DEL DÍA: Q 1482.40	**Aprobado**
CP-REP-06	REQ-0006, Alta	**Inventario Existente.** Datos: Base del flujo completo	P001 84, P002 48, P003 65, P004 46	Total de registros: 4	**Aprobado**
CP-REP-07	REQ-0006, Media	**Productos vendidos con rango sin ventas.** Datos: Desde 2099-01-01	Sin registros	Total de registros: 0	**Aprobado**
CP-REP-08	REQ-0006, Media	**Reportes en base vacía.** Datos: Ventas del día sin facturas	Informa que no hay datos, sin errores	Total de registros: 0 / TOTAL VENDIDO DEL DÍA: Q 0.00	**Aprobado**

11.8 Interfaz y persistencia (REQ-0007, REQ-0008)

ID	Req. / Prior.	Descripción y datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-UI-01	REQ-0007, Media	**Opción de menú inválida.** Datos: Opción 9 en el menú principal	Mensaje de valor inválido y vuelve a mostrar el menú	Valor inválido. Ingrese un número entero entre 0 y 6.	**Aprobado**
CP-UI-02	REQ-0007, Media	**Letras donde se espera un número.** Datos: Texto 'abc' en el menú	Mensaje claro, no se cierra el programa	Valor inválido. Ingrese un número entero entre 0 y 6.	**Aprobado**
CP-PER-01	REQ-0008, Alta	**Los datos persisten al cerrar y volver a abrir.** Datos: Sesión 1 registra V003; sesión 2 consulta	V003 sigue en la lista	Total de registros: 4	**Aprobado**

11.9 Integridad de datos y rendimiento (REQ-0003 a REQ-0009)

ID	Req. / Prior.	Descripción y datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-INT-01	REQ-0009, Alta	**Verificación de integridad de la BD.** Datos: PRAGMA integrity_check	Devuelve ok	integrity_check = ok	**Aprobado**
CP-INT-02	REQ-0009, Alta	**Verificación de llaves foráneas.** Datos: PRAGMA foreign_key_check	Sin filas (ninguna referencia rota)	foreign_key_check = 0 filas	**Aprobado**
CP-INT-03	REQ-0003, Alta	**Inventario nunca negativo.** Datos: SELECT con existencia < 0	0 productos	0 productos con existencia negativa	**Aprobado**
CP-INT-04	REQ-0004, Alta	**IVA y total cuadran en todas las facturas.** Datos: subtotal + IVA = total e IVA = 12 % del subtotal	0 facturas con diferencia	0 facturas con diferencia	**Aprobado**
CP-INT-05	REQ-0004, Alta	**El subtotal de cada factura es la suma de sus líneas.** Datos: Suma de DetalleFactura vs Factura.subtotal	0 diferencias	0 facturas con diferencia	**Aprobado**
CP-INT-06	REQ-0005, Alta	**El saldo de cada factura = total - pagos.** Datos: Comparar saldo registrado con total menos pagos	0 diferencias	0 facturas con diferencia	**Aprobado**

ID	Req. / Prior.	Descripción y datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-INT-07	REQ-0009, Alta	**La BD rechaza un pago de una factura inexistente.** Datos: INSERT Pago con factura 999	Error de llave foránea	FOREIGN KEY constraint failed	**Aprobado**
CP-INT-08	REQ-0003, Alta	**La BD rechaza existencia negativa.** Datos: UPDATE Producto SET existencia = -1	Error de restricción CHECK	CHECK constraint failed: existencia >= 0	**Aprobado**
CP-INT-09	REQ-0004, Media	**La BD rechaza una condición de pago inválida.** Datos: INSERT Factura con condición 'Otro'	Error de restricción CHECK	CHECK constraint failed: condicion IN ('Contado','Crédito')	**Aprobado**
CP-INT-10	REQ-0001, Media	**La BD impide borrar un vendedor con clientes.** Datos: DELETE de V001 (tiene clientes)	Error de llave foránea	FOREIGN KEY constraint failed	**Aprobado**
CP-INT-11	REQ-0004, Media	**La BD rechaza una línea de factura con cantidad 0.** Datos: INSERT DetalleFactura con cantidad 0	Error de restricción CHECK	CHECK constraint failed: cantidad > 0	**Aprobado**
CP-REN-01	REQ-0008, Media	**Tiempo de respuesta de reportes con 5,000 facturas.** Datos: Se agregan 5,000 facturas con detalle y pago y se corren 3 reportes	Cada reporte tarda menos de 3 segundos	Ventas del Día (5,004 facturas): 0.01 s / Productos Vendidos (5,004 facturas): 0.02 s / Pagos del Día (5,004 pagos): 0.01 s	**Aprobado**

11.10 Pruebas sobre el código (caja blanca)

Además de los casos por módulo, se analizó el código fuente (`main.cpp`, 1,018 líneas) con tres herramientas. Los resultados son los siguientes:

Prueba sobre el código	Herramienta	Resultado esperado	Resultado obtenido	Estado
Compilación sin advertencias	g++ con -Wall -Wextra	0 advertencias	0 advertencias, 0 errores	Aprobado
Errores de memoria (desbordes, accesos inválidos, uso después de liberar)	AddressSanitizer	Ninguno en los 90 casos	Ninguno detectado	Aprobado
Comportamiento indefinido (desbordes de enteros, conversiones)	UndefinedBehaviorSanitizer	Ninguno en los 90 casos	Ninguno detectado	Aprobado
Cobertura de líneas ejecutadas por las pruebas	gcov	Al menos 90 %	96.3 % (626 de 650 líneas ejecutables)	Aprobado
Consultas SQL parametrizadas (sin concatenar datos del usuario)	Revisión del código y CP-COD-01	Ningún dato del usuario se une al texto SQL	Todas las consultas usan parámetros; el ataque de CP-COD-01 se guarda como texto	Aprobado

Líneas no cubiertas (24 de 650). Son manejadores de errores que no se pueden provocar desde el uso normal: fallo al abrir o crear el archivo de la base de datos, error interno de SQLite, el recorte de texto con ~ en columnas muy angostas y algunos mensajes de factura inexistente que el menú ya impide alcanzar. Se revisaron a mano y su lógica es simple (mostrar un mensaje y volver).

A continuación se listan los casos de prueba que ejercitan ramas específicas del código: transacciones con ROLLBACK, filtros por vendedor, validaciones en las modificaciones y cierre ordenado ante una entrada interrumpida.

ID	Req. / Prior.	Descripción y datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-COD-01	REQ-0009, Alta	**Inyección SQL: texto con comillas y DROP TABLE como nombre de cliente.** Datos: Nombre: x'); DROP TABLE Cliente;--	Se guarda como texto normal; la tabla Cliente sigue existiendo	Cliente registrado correctamente. [Valor en BD: 1, x'); DROP TABLE Cliente;--]	**Aprobado**
CP-COD-02	REQ-0004, Alta	**Transacción: falla a mitad de la factura (se simula con un trigger que aborta el INSERT del detalle).** Datos: Factura contado de 10 x P001 con el fallo simulado	ROLLBACK: no queda factura, ni pago, ni cambio en el inventario	No se pudo generar la factura. No se realizó ningún cambio. [Valor en BD: 0, 0, 120]	**Aprobado**
CP-COD-03	REQ-0008, Media	**Fin inesperado de la entrada (stdin cerrado).** Datos: Programa sin ningún dato de entrada	Cierra de forma ordenada, sin colgarse ni fallar	[Fin de la entrada. Cerrando el sistema]	**Aprobado**
CP-COD-04	REQ-0003, Media	**Modificar producto con costo en texto.** Datos: Costo: abc	Mensaje de valor inválido; sin cambios	Valor numérico inválido. No se realizaron cambios. [Valor en BD: 18.5]	**Aprobado**
CP-COD-05	REQ-0003, Media	**Modificar producto con costo negativo.** Datos: Costo: -5	Rechaza; sin cambios	Los valores no pueden ser negativos. No se realizaron cambios. [Valor en BD: 18.5]	**Aprobado**
CP-COD-06	REQ-0003, Baja	**Modificar un producto que no existe.** Datos: Código P999	Mensaje de no encontrado	No existe un producto con ese código.	**Aprobado**
CP-COD-07	REQ-0002, Alta	**Vendedor intenta modificar un cliente de otro vendedor.** Datos: V001 modifica el cliente 2 (de V002)	Rechaza: no está en su cartera	No existe un cliente con ese ID en su cartera.	**Aprobado**
CP-COD-08	REQ-0002, Alta	**Vendedor intenta ver el historial de un cliente ajeno.** Datos: V001 consulta el historial del cliente 2	Rechaza: no está en su cartera	No existe un cliente con ese ID en su cartera.	**Aprobado**
CP-COD-09	REQ-0004, Media	**Producto agotado dentro de la misma factura.** Datos: P003 por 80 unidades y luego P003 otra vez	Avisa que ya no hay existencia disponible	Sin existencia disponible de Cloro 1 galón.	**Aprobado**
CP-COD-10	REQ-0007, Baja	**Respuesta inválida en la confirmación.** Datos: Responde 'x' y luego N	Pide responder S o N	Responda S o N. / Factura cancelada.	**Aprobado**

PARTE 3: RESULTADOS Y CONCLUSIONES

12. Resumen de resultados

Se ejecutaron **90 casos de prueba**. De ellos, **90 fueron aprobados** (100 %) y **0 fallidos**. A continuación se muestra el detalle por módulo y por prioridad.

Módulo	Casos	Aprobados	Fallidos	% aprobado
Acceso y seguridad	6	6	0	100 %
Empleados	9	9	0	100 %
Clientes	8	8	0	100 %

Módulo	Casos	Aprobados	Fallidos	% aprobado
Productos e inventario	9	9	0	100 %
Facturación	14	14	0	100 %
Pagos	11	11	0	100 %
Reportes	8	8	0	100 %
Interfaz y persistencia	3	3	0	100 %
Integridad y rendimiento	12	12	0	100 %
Pruebas sobre el código	10	10	0	100 %
TOTAL	**90**	**90**	**0**	**100 %**

Prioridad	Casos	Aprobados	Fallidos
Alta	51	51	0
Media	34	34	0
Baja	5	5	0

13. Matriz de trazabilidad

La matriz relaciona cada requerimiento del catálogo con los casos que lo verifican. Todos los requerimientos tienen al menos una prueba.

Requerimiento	Descripción	Casos	Casos que lo verifican	Estado
REQ-0001	Empleados y cartera de vendedores	10	EMP-01, EMP-02, EMP-03, EMP-04, EMP-05, EMP-06, EMP-07, EMP-08, EMP-09, INT-10	Cubierto
REQ-0002	Clientes e historial	10	CLI-01, CLI-02, CLI-03, CLI-04, CLI-05, CLI-06, CLI-07, CLI-08, COD-07, COD-08	Cubierto
REQ-0003	Productos e inventario	14	PRO-01, PRO-02, PRO-03, PRO-04, PRO-05, PRO-06, PRO-07, PRO-08, PRO-09, COD-04, COD-05, COD-06, INT-03, INT-08	Cubierto
REQ-0004	Facturas, detalle e IVA	20	FAC-01, FAC-02, FAC-03, FAC-04, FAC-05, FAC-06, FAC-07, FAC-08, FAC-09, FAC-10, FAC-11, FAC-12, FAC-13, FAC-14, COD-02, COD-09, INT-04, INT-05, INT-09, INT-11	Cubierto
REQ-0005	Pagos y saldo	12	PAG-01, PAG-02, PAG-03, PAG-04, PAG-05, PAG-06, PAG-07, PAG-08, PAG-09, PAG-10, PAG-11, INT-06	Cubierto
REQ-0006	Seis reportes	8	REP-01, REP-02, REP-03, REP-04, REP-05, REP-06, REP-07, REP-08	Cubierto
REQ-0007	Interfaz sencilla	3	UI-01, UI-02, COD-10	Cubierto
REQ-0008	Rendimiento y disponibilidad	3	PER-01, COD-03, REN-01	Cubierto
REQ-0009	Autenticación y validación	10	ACC-01, ACC-02, ACC-03, ACC-04, ACC-05, ACC-06, COD-01, INT-01, INT-02, INT-07	Cubierto

Requerimiento	Descripción	Casos	Casos que lo verifican	Estado
REQ-0010	Crecimiento y equipos actuales	0	Cubierto por inspección: el sistema usa C++17 y SQLite, sin servidor, y corre en equipos Linux y Windows comunes. La base de datos usa tablas separadas que permiten agregar sucursales o usuarios después.	Cubierto

14. Defectos

14.1 Defectos en la ejecución final

En la ejecución final de los 90 casos **no se encontraron defectos**: todos los resultados obtenidos coinciden con los esperados.

14.2 Defectos encontrados y corregidos durante el desarrollo

Durante las pruebas informales previas a esta ejecución se detectaron dos problemas. Ambos se corrigieron antes de ejecutar el plan, y los casos actuales los verifican.

ID	Defecto	Severidad	Corrección	Caso que lo verifica
DEF-01	En los reportes, los nombres y direcciones largos se cortaban con el símbolo ~ y se perdía información.	Media	Se agregó una segunda línea de detalle bajo cada registro en los reportes de Clientes y Vendedores.	CP-REP-01, CP-REP-02
DEF-02	Al duplicar un dato único (código, DPI o NIT), el sistema mostraba el mensaje técnico de SQLite, difícil de entender.	Media	Se creó una función que traduce el error a un mensaje claro, por ejemplo "Ya existe un cliente con ese NIT".	CP-EMP-03, CP-EMP-04, CP-CLI-02, CP-PRO-02

14.3 Observaciones y limitaciones conocidas

Estos puntos no son defectos, sino decisiones de diseño o límites del alcance que conviene tener presentes:

- La clave de un Vendedor es su número de DPI. Es simple y suficiente para el proyecto; en una versión comercial se recomienda una clave propia cifrada.
- El programa avisa cuando el precio es menor al costo, pero permite guardarlo, porque la gerencia puede necesitar vender con pérdida en una liquidación.
- No existe la opción de eliminar registros: los datos con facturas no deben borrarse. La base de datos lo refuerza con llaves foráneas (CP-INT-10).
- Es un sistema de un solo usuario por equipo. No se probó el acceso simultáneo de varias personas a la misma base de datos.

15. Verificación de los criterios de salida

Criterio (sección 7.4)	Resultado	Cumple
100 % de los casos de prioridad Alta aprobados	51 de 51 aprobados	Sí
Sin defectos Críticos o Altos abiertos	0 defectos abiertos	Sí

Criterio (sección 7.4)	Resultado	Cumple
Código sin errores de memoria y con cobertura de al menos 90 %	AddressSanitizer y UBSan sin hallazgos; cobertura 96.3 % (sección 11.10)	Sí
Los seis reportes cuadran con los datos registrados	Ventas Q 2,190.72 y pagos Q 1,482.40 coinciden con las 4 facturas y 4 pagos (CP-REP-04, CP-REP-05)	Sí
integrity_check = ok y foreign_key_check sin filas	ok y 0 filas (CP-INT-01, CP-INT-02)	Sí

16. Conclusiones

- El sistema superó los 90 casos de prueba, lo que da confianza en que cumple los requerimientos documentados desde el Entregable 3.
- Los cálculos de dinero son correctos: el IVA del 12 % se redondea bien a dos decimales (CP-FAC-12) y el saldo de cada factura siempre coincide con el total menos los pagos (CP-INT-06).
- El análisis del código con herramientas no encontró errores de memoria ni comportamiento indefinido, el programa compila sin advertencias y las pruebas ejecutan el 96.3 % de sus líneas (sección 11.10). Las consultas SQL parametrizadas resisten la inyección SQL (CP-COD-01) y las transacciones revierten todo ante un fallo (CP-COD-02).
- El control de inventario es confiable: no permite vender más de lo que hay, ni siquiera sumando varias líneas del mismo producto (CP-FAC-11), y la base de datos impide existencias negativas (CP-INT-08).
- La seguridad básica funciona: cada rol ve solo sus opciones, un vendedor solo atiende a sus clientes y los Repartidores no entran al sistema.
- Las pruebas de integridad muestran que, aun con entradas erróneas, la base de datos quedó consistente. Las transacciones evitan facturas a medias.
- El rendimiento es más que suficiente para la empresa: con 5,000 facturas, los reportes responden en menos de una décima de segundo (CP-REN-01).

Recomendaciones

- Repetir este plan completo cada vez que se modifique el código, para detectar errores nuevos en funciones que ya funcionaban.
- En una siguiente versión, agregar claves propias para los vendedores y un respaldo (copia de seguridad) automático del archivo de la base de datos.
- Si la empresa crece a varias cajas o sucursales, probar el acceso simultáneo o migrar a un servidor de base de datos.

10. Repositorio y ejecución del programa

El código fuente se entrega en un repositorio con la siguiente estructura, junto con la base de datos de demostración y la documentación:

Carpeta / archivo	Contenido
`src/main.cpp`	Código fuente completo del sistema (C++17, un solo archivo de unas 1,000 líneas)
`db/chapina_demo.db`	Base de datos SQLite con datos de ejemplo
`docs/`	Este documento y el diagrama de Gantt
`pruebas/`	Resultados de los 90 casos de prueba
`README.md` **y** `Makefile`	Instrucciones de instalación, compilación y uso

10.1 Requisitos y compilación

Se necesita el compilador g++ (con C++17) y la biblioteca SQLite3. En Ubuntu o WSL se instalan con `sudo apt install g++ libsqlite3-dev`; en Windows se usa MSYS2 con `pacman -S mingw-w64-ucrt-x86_64-gcc mingw-w64-ucrt-x86_64-sqlite3`. Para compilar:

```
g++ -std=c++17 -Wall -Wextra -o chapina src/main.cpp -lsqlite3
```

10.2 Ejecución

- Con datos de ejemplo: `./chapina db/chapina_demo.db`
- Con una base de datos nueva y vacía (se crea sola): `./chapina`

10.3 Usuarios de la demostración

Rol	Usuario	Clave
Administrador	admin	admin123
Vendedor	V001 (Carlos Méndez López)	su DPI: 2456789010101
Vendedor	V002 (Ana Lucía Ruiz)	su DPI: 2987654320101

11. Conclusiones

- El sistema cumple los diez requerimientos documentados: gestiona empleados, clientes, productos, facturas y pagos, y emite los seis reportes solicitados.
- La base de datos SQLite, con llaves foráneas, restricciones y transacciones, protege la integridad de la información aun ante errores de captura.
- Las 90 pruebas del Entregable 8 se aprobaron, incluidos los cálculos de IVA y saldos, el control de inventario y los reportes.
- La propuesta comercial y el estudio de factibilidad muestran que el proyecto es viable técnica y económicamente para una empresa de este tamaño.
- Como mejoras futuras se recomiendan claves propias para los vendedores, copias de seguridad automáticas y soporte para varias cajas o sucursales.

Anexo A. Código fuente: main.cpp (C++17 + SQLite3)

```
// ============================================================================
//  Sistema de Facturación - Representaciones "Chapina", C.A.
//  Autor : Werner Cifuentes
//  Fecha : 07/10/2026
//  Compilar: g++ -std=c++17 -o chapina main.cpp -lsqlite3
//  Ejecutar: ./chapina            (usa el archivo chapina.db)
//            ./chapina otra.db    (usa otro archivo de base de datos)
// ============================================================================
#include <sqlite3.h>

#ifdef _WIN32
#include <io.h>
#include <windows.h>
#define ENTRADA_ES_TECLADO() _isatty(_fileno(stdin))
#else
#include <unistd.h>
#define ENTRADA_ES_TECLADO() isatty(STDIN_FILENO)
#endif

#include <climits>
#include <cmath>
#include <cstdlib>
#include <iomanip>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>

using namespace std;

typedef vector<string> Fila;
typedef vector<Fila> Tabla;

static sqlite3* db = nullptr;

// ------------------------------------------------------------------ Sesión ---
enum Rol { ADMIN, VENDEDOR };
static Rol rolActual = ADMIN;
static string usuarioActual;
static string nombreUsuario;

static const double TASA_IVA = 0.12;  // IVA 12 %

// ============================================================================
//  1. UTILIDADES DE ENTRADA / SALIDA
// ============================================================================
static string recortar(const string& s) {
    size_t a = s.find_first_not_of(" \t\r\n");
    if (a == string::npos) return "";
    size_t b = s.find_last_not_of(" \t\r\n");
    return s.substr(a, b - a + 1);
}

static string leer(const string& mensaje) {
    cout << mensaje;
    string s;
    if (!getline(cin, s)) {
        cout << "\n[Fin de la entrada. Cerrando el sistema]\n";
        if (db) sqlite3_close(db);
        exit(0);
    }
    s = recortar(s);
    // Si la entrada viene de un archivo (pruebas automáticas), se muestra lo "escrito"
    // para que la salida se lea igual que una sesión real en pantalla.
    if (!ENTRADA_ES_TECLADO()) cout << s << "\n";
    return s;
}

static string leerObligatorio(const string& mensaje) {
    while (true) {
        string s = leer(mensaje);
        if (!s.empty()) return s;
        cout << "  Este dato es obligatorio.\n";
    }
}

static string leerOpcional(const string& mensaje, const string& actual) {
    string s = leer(mensaje + " [" + actual + "]: ");
    return s.empty() ? actual : s;
```

Anexo A. Código fuente: main.cpp (C++17 + SQLite3)

```
}

static int leerEntero(const string& mensaje, int minimo, int maximo) {
    while (true) {
        string s = leer(mensaje);
        try {
            size_t pos = 0;
            long v = stol(s, &pos);
            if (pos == s.size() && v >= minimo && v <= maximo) return (int)v;
        } catch (...) {
        }
        cout << "  Valor inválido. Ingrese un número entero";
        if (maximo != INT_MAX) cout << " entre " << minimo << " y " << maximo;
        else cout << " mayor o igual a " << minimo;
        cout << ".\n";
    }
}

static double leerDecimal(const string& mensaje, double minimo) {
    while (true) {
        string s = leer(mensaje);
        try {
            size_t pos = 0;
            double v = stod(s, &pos);
            if (pos == s.size() && v >= minimo) return v;
        } catch (...) {
        }
        cout << "  Valor inválido. Ingrese un número mayor o igual a " << minimo << ".\n";
    }
}

static bool confirmar(const string& mensaje) {
    while (true) {
        string s = leer(mensaje + " (S/N): ");
        if (s == "S" || s == "s") return true;
        if (s == "N" || s == "n") return false;
        cout << "  Responda S o N.\n";
    }
}

static string num2(double v) {
    char buf[64];
    snprintf(buf, sizeof(buf), "%.2f", v);
    return buf;
}

static double redondear(double v) { return round(v * 100.0) / 100.0; }

static string dinero(const string& s) {
    try {
        return num2(stod(s));
    } catch (...) {
        return s;
    }
}

static void titulo(const string& t) {
    cout << "\n--- " << t << " ---\n";
}

// Longitud de una cadena UTF-8 en caracteres (no en bytes), para alinear tablas.
static size_t largo(const string& s) {
    size_t n = 0;
    for (unsigned char c : s)
        if ((c & 0xC0) != 0x80) n++;
    return n;
}

static string ajustar(const string& texto, size_t ancho, bool derecha) {
    string t = texto;
    if (largo(t) > ancho) {
        size_t n = 0, i = 0;
        while (i < t.size() && n + 1 < ancho) {
            i++;
            while (i < t.size() && (((unsigned char)t[i] & 0xC0) == 0x80)) i++;
            n++;
        }
        t = t.substr(0, i) + "~";
    }
```

Anexo A. Código fuente: main.cpp (C++17 + SQLite3)

```
    size_t l = largo(t);
    string relleno(ancho > l ? ancho - l : 0, ' ');
    return derecha ? relleno + t : t + relleno;
}

// 'detalle' (opcional): texto que se imprime en una segunda línea bajo cada fila,
// útil para datos largos (direcciones, correos) que no caben en una columna.
static void imprimirTabla(const vector<string>& enc, const vector<int>& anch, const Tabla& filas,
                          const vector<bool>& der = {}, const vector<string>& detalle = {}) {
    int total = 0;
    for (int a : anch) total += a + 2;
    string sep(total, '-');
    cout << sep << "\n";
    for (size_t i = 0; i < enc.size(); i++)
        cout << ajustar(enc[i], anch[i], i < der.size() && der[i]) << "  ";
    cout << "\n" << sep << "\n";
    for (size_t k = 0; k < filas.size(); k++) {
        const Fila& f = filas[k];
        for (size_t i = 0; i < f.size() && i < anch.size(); i++)
            cout << ajustar(f[i], anch[i], i < der.size() && der[i]) << "  ";
        cout << "\n";
        if (k < detalle.size() && !detalle[k].empty()) cout << "      " << detalle[k] << "\n";
    }
    cout << sep << "\n";
    cout << "Total de registros: " << filas.size() << "\n";
}

// ============================================================================
//  2. ACCESO A BASE DE DATOS (SQLite)
// ============================================================================
// Traduce los errores técnicos de SQLite a mensajes comprensibles para el usuario.
static string traducirError(const string& m) {
    if (m.find("UNIQUE constraint failed: Empleado.codigo") != string::npos) return "Ya existe un empleado con ese cód
    if (m.find("UNIQUE constraint failed: Empleado.dpi") != string::npos) return "Ya existe un empleado con ese DPI.";
    if (m.find("UNIQUE constraint failed: Cliente.nit") != string::npos) return "Ya existe un cliente con ese NIT.";
    if (m.find("UNIQUE constraint failed: Producto.codigo") != string::npos) return "Ya existe un producto con ese cód
    if (m.find("FOREIGN KEY") != string::npos) return "El dato hace referencia a un registro que no existe.";
    if (m.find("CHECK constraint failed") != string::npos) return "Un valor está fuera del rango permitido.";
    return m;
}

// Ejecuta una sentencia con parámetros (?) y, opcionalmente, devuelve las filas.
static bool ejecutar(const string& sql, const Fila& params = {}, Tabla* salida = nullptr) {
    sqlite3_stmt* st = nullptr;
    if (sqlite3_prepare_v2(db, sql.c_str(), -1, &st, nullptr) != SQLITE_OK) {
        cout << "  Error interno de base de datos: " << sqlite3_errmsg(db) << "\n";
        return false;
    }
    for (size_t i = 0; i < params.size(); i++)
        sqlite3_bind_text(st, (int)i + 1, params[i].c_str(), -1, SQLITE_TRANSIENT);

    int cols = sqlite3_column_count(st);
    int rc;
    while ((rc = sqlite3_step(st)) == SQLITE_ROW) {
        if (salida) {
            Fila f;
            for (int c = 0; c < cols; c++) {
                const unsigned char* t = sqlite3_column_text(st, c);
                f.push_back(t ? reinterpret_cast<const char*>(t) : "");
            }
            salida->push_back(f);
        }
    }
    bool ok = (rc == SQLITE_DONE);
    if (!ok) cout << "  No se pudo completar la operación: " << traducirError(sqlite3_errmsg(db)) << "\n";
    sqlite3_finalize(st);
    return ok;
}

static void crearEsquema() {
    const char* esquema = R"SQL(
PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS Empleado (
    codigo    TEXT PRIMARY KEY,
    dpi       TEXT NOT NULL UNIQUE,
    nombre    TEXT NOT NULL,
    telefono  TEXT,
    direccion TEXT,
```

Anexo A. Código fuente: main.cpp (C++17 + SQLite3)

```
    email     TEXT,
    tipo      TEXT NOT NULL CHECK (tipo IN ('Vendedor','Repartidor'))
);

CREATE TABLE IF NOT EXISTS Cliente (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre          TEXT NOT NULL,
    nit             TEXT NOT NULL UNIQUE,
    telefono        TEXT,
    direccion       TEXT,
    contacto        TEXT,
    email           TEXT,
    web             TEXT,
    vendedor_codigo TEXT NOT NULL REFERENCES Empleado(codigo)
);

CREATE TABLE IF NOT EXISTS Producto (
    codigo     TEXT PRIMARY KEY,
    nombre     TEXT NOT NULL,
    costo      REAL NOT NULL CHECK (costo >= 0),
    precio     REAL NOT NULL CHECK (precio >= 0),
    existencia INTEGER NOT NULL CHECK (existencia >= 0)
);

CREATE TABLE IF NOT EXISTS Factura (
    correlativo       INTEGER PRIMARY KEY AUTOINCREMENT,
    serie             TEXT NOT NULL,
    fecha             TEXT NOT NULL,
    cliente_id        INTEGER NOT NULL REFERENCES Cliente(id),
    vendedor_codigo   TEXT NOT NULL REFERENCES Empleado(codigo),
    condicion         TEXT NOT NULL CHECK (condicion IN ('Contado','Crédito')),
    plazo             INTEGER NOT NULL DEFAULT 0,
    fecha_vencimiento TEXT NOT NULL,
    subtotal          REAL NOT NULL,
    iva               REAL NOT NULL,
    total             REAL NOT NULL
);

CREATE TABLE IF NOT EXISTS DetalleFactura (
    id                  INTEGER PRIMARY KEY AUTOINCREMENT,
    factura_correlativo INTEGER NOT NULL REFERENCES Factura(correlativo),
    producto_codigo     TEXT NOT NULL REFERENCES Producto(codigo),
    cantidad            INTEGER NOT NULL CHECK (cantidad > 0),
    precio_unitario     REAL NOT NULL,
    subtotal            REAL NOT NULL
);

CREATE TABLE IF NOT EXISTS Pago (
    correlativo         INTEGER PRIMARY KEY AUTOINCREMENT,
    factura_correlativo INTEGER NOT NULL REFERENCES Factura(correlativo),
    fecha               TEXT NOT NULL,
    monto               REAL NOT NULL CHECK (monto > 0),
    saldo               REAL NOT NULL CHECK (saldo >= 0)
);
)SQL";
    char* err = nullptr;
    if (sqlite3_exec(db, esquema, nullptr, nullptr, &err) != SQLITE_OK) {
        cerr << "No se pudo crear la base de datos: " << (err ? err : "") << "\n";
        sqlite3_free(err);
        sqlite3_close(db);
        exit(1);
    }
}

static string saldoFactura(const string& correlativo) {
    Tabla t;
    ejecutar("SELECT ROUND(total - COALESCE((SELECT SUM(monto) FROM Pago WHERE factura_correlativo = f.correlativo), 0
             "FROM Factura f WHERE correlativo = ?",
             {correlativo}, &t);
    return t.empty() ? "" : t[0][0];
}

// ============================================================================
//  3. MÓDULO DE EMPLEADOS  (Vendedores y Repartidores)
// ============================================================================
static void listarEmpleados() {
    Tabla t;
    ejecutar("SELECT codigo, dpi, nombre, telefono, email, tipo FROM Empleado ORDER BY codigo", {}, &t);
    imprimirTabla({"Código", "DPI", "Nombre", "Teléfono", "E-mail", "Tipo"}, {8, 14, 24, 10, 24, 10}, t);
```

Anexo A. Código fuente: main.cpp (C++17 + SQLite3)

```
}

static void registrarEmpleado() {
    titulo("Registrar empleado");
    string codigo = leerObligatorio("Código de empleado : ");
    string dpi = leerObligatorio("DPI                : ");
    string nombre = leerObligatorio("Nombre             : ");
    string tel = leer("Teléfono          : ");
    string dir = leer("Dirección         : ");
    string mail = leer("E-mail            : ");
    int tipo = leerEntero("Tipo (1=Vendedor, 2=Repartidor): ", 1, 2);
    if (ejecutar("INSERT INTO Empleado(codigo,dpi,nombre,telefono,direccion,email,tipo) VALUES(?,?,?,?,?,?,?)",
                 {codigo, dpi, nombre, tel, dir, mail, tipo == 1 ? "Vendedor" : "Repartidor"}))
        cout << "Empleado registrado correctamente.\n";
}

static void modificarEmpleado() {
    titulo("Modificar empleado");
    listarEmpleados();
    string codigo = leerObligatorio("Código del empleado a modificar: ");
    Tabla t;
    ejecutar("SELECT nombre, telefono, direccion, email FROM Empleado WHERE codigo = ?", {codigo}, &t);
    if (t.empty()) {
        cout << "No existe un empleado con ese código.\n";
        return;
    }
    cout << "(Presione Enter para conservar el valor actual)\n";
    string nombre = leerOpcional("Nombre   ", t[0][0]);
    string tel = leerOpcional("Teléfono ", t[0][1]);
    string dir = leerOpcional("Dirección", t[0][2]);
    string mail = leerOpcional("E-mail   ", t[0][3]);
    if (ejecutar("UPDATE Empleado SET nombre=?, telefono=?, direccion=?, email=? WHERE codigo=?",
                 {nombre, tel, dir, mail, codigo}))
        cout << "Empleado actualizado correctamente.\n";
}

// ============================================================================
//  4. MÓDULO DE CLIENTES
// ============================================================================
static void listarClientes(bool soloPropios) {
    Tabla t;
    string sql = "SELECT c.id, c.nombre, c.nit, c.telefono, c.contacto, c.vendedor_codigo FROM Cliente c";
    Fila p;
    if (soloPropios) {
        sql += " WHERE c.vendedor_codigo = ?";
        p.push_back(usuarioActual);
    }
    sql += " ORDER BY c.id";
    ejecutar(sql, p, &t);
    imprimirTabla({"ID", "Nombre", "NIT", "Teléfono", "Contacto", "Vendedor"}, {4, 24, 11, 10, 18, 8}, t,
                  {true, false, false, false, false, false});
}

static void registrarCliente() {
    titulo("Registrar cliente");
    string nombre = leerObligatorio("Nombre           : ");
    string nit = leerObligatorio("NIT              : ");
    string tel = leer("Teléfono         : ");
    string dir = leer("Dirección        : ");
    string contacto = leer("Persona contacto : ");
    string mail = leer("E-mail           : ");
    string web = leer("Página web       : ");

    string vendedor;
    if (rolActual == VENDEDOR) {
        vendedor = usuarioActual;
        cout << "Vendedor asignado: " << vendedor << " (usted)\n";
    } else {
        Tabla v;
        ejecutar("SELECT codigo, nombre FROM Empleado WHERE tipo='Vendedor' ORDER BY codigo", {}, &v);
        imprimirTabla({"Código", "Vendedor"}, {8, 30}, v);
        vendedor = leerObligatorio("Código del vendedor asignado: ");
        Tabla ok;
        ejecutar("SELECT codigo FROM Empleado WHERE codigo=? AND tipo='Vendedor'", {vendedor}, &ok);
        if (ok.empty()) {
            cout << "Ese código no corresponde a un Vendedor registrado.\n";
            return;
        }
    }
```

Anexo A. Código fuente: main.cpp (C++17 + SQLite3)

```
    if (ejecutar("INSERT INTO Cliente(nombre,nit,telefono,direccion,contacto,email,web,vendedor_codigo) VALUES(?,?,?,?
                 {nombre, nit, tel, dir, contacto, mail, web, vendedor}))
        cout << "Cliente registrado correctamente.\n";
}

static void modificarCliente() {
    titulo("Modificar cliente");
    listarClientes(rolActual == VENDEDOR);
    string id = leerObligatorio("ID del cliente a modificar: ");
    Tabla t;
    string sql = "SELECT nombre, telefono, direccion, contacto, email, web FROM Cliente WHERE id = ?";
    Fila p = {id};
    if (rolActual == VENDEDOR) {
        sql += " AND vendedor_codigo = ?";
        p.push_back(usuarioActual);
    }
    ejecutar(sql, p, &t);
    if (t.empty()) {
        cout << "No existe un cliente con ese ID en su cartera.\n";
        return;
    }
    cout << "(Presione Enter para conservar el valor actual)\n";
    string nombre = leerOpcional("Nombre          ", t[0][0]);
    string tel = leerOpcional("Teléfono        ", t[0][1]);
    string dir = leerOpcional("Dirección       ", t[0][2]);
    string con = leerOpcional("Persona contacto", t[0][3]);
    string mail = leerOpcional("E-mail          ", t[0][4]);
    string web = leerOpcional("Página web      ", t[0][5]);
    if (ejecutar("UPDATE Cliente SET nombre=?, telefono=?, direccion=?, contacto=?, email=?, web=? WHERE id=?",
                 {nombre, tel, dir, con, mail, web, id}))
        cout << "Cliente actualizado correctamente.\n";
}

static void consultarClientes() {
    titulo("Clientes registrados");
    listarClientes(rolActual == VENDEDOR);
}

static void historialCliente() {
    titulo("Historial de un cliente");
    listarClientes(rolActual == VENDEDOR);
    string id = leerObligatorio("ID del cliente: ");
    Tabla c;
    string sql = "SELECT nombre, nit FROM Cliente WHERE id = ?";
    Fila p = {id};
    if (rolActual == VENDEDOR) {
        sql += " AND vendedor_codigo = ?";
        p.push_back(usuarioActual);
    }
    ejecutar(sql, p, &c);
    if (c.empty()) {
        cout << "No existe un cliente con ese ID en su cartera.\n";
        return;
    }
    cout << "\nCliente: " << c[0][0] << "   NIT: " << c[0][1] << "\n\nFacturas:\n";
    Tabla f;
    ejecutar("SELECT f.correlativo, f.fecha, f.condicion, f.total, "
             "ROUND(f.total - COALESCE((SELECT SUM(monto) FROM Pago WHERE factura_correlativo=f.correlativo),0),2) "
             "FROM Factura f WHERE f.cliente_id = ? ORDER BY f.correlativo",
             {id}, &f);
    for (Fila& r : f) {
        r[3] = dinero(r[3]);
        r[4] = dinero(r[4]);
    }
    imprimirTabla({"Factura", "Fecha", "Condición", "Total", "Saldo"}, {8, 10, 10, 12, 12}, f,
                  {true, false, false, true, true});

    cout << "\nPagos realizados:\n";
    Tabla pg;
    ejecutar("SELECT p.correlativo, p.factura_correlativo, p.fecha, p.monto, p.saldo FROM Pago p "
             "JOIN Factura f ON f.correlativo = p.factura_correlativo WHERE f.cliente_id = ? ORDER BY p.correlativo",
             {id}, &pg);
    for (Fila& r : pg) {
        r[3] = dinero(r[3]);
        r[4] = dinero(r[4]);
    }
    imprimirTabla({"Pago", "Factura", "Fecha", "Monto", "Saldo"}, {6, 8, 10, 12, 12}, pg,
                  {true, true, false, true, true});
}
```

Anexo A. Código fuente: main.cpp (C++17 + SQLite3)

```
475
476 // ============================================================================
477 //  5. MÓDULO DE PRODUCTOS E INVENTARIO
478 // ============================================================================
479 static void listarProductos() {
480     Tabla t;
481     ejecutar("SELECT codigo, nombre, costo, precio, existencia FROM Producto ORDER BY codigo", {}, &t);
482     for (Fila& r : t) {
483         r[2] = dinero(r[2]);
484         r[3] = dinero(r[3]);
485     }
486     imprimirTabla({"Código", "Producto", "Costo", "Precio", "Existencia"}, {8, 28, 10, 10, 10}, t,
487                   {false, false, true, true, true});
488 }
489
490 static void registrarProducto() {
491     titulo("Registrar producto");
492     string codigo = leerObligatorio("Código     : ");
493     string nombre = leerObligatorio("Nombre     : ");
494     double costo = leerDecimal("Costo (Q)  : ", 0);
495     double precio = leerDecimal("Precio (Q) : ", 0);
496     if (precio < costo) cout << "  Aviso: el precio de venta es menor al costo.\n";
497     int exist = leerEntero("Existencia : ", 0, INT_MAX);
498     if (ejecutar("INSERT INTO Producto(codigo,nombre,costo,precio,existencia) VALUES(?,?,?,?,?)",
499                  {codigo, nombre, num2(costo), num2(precio), to_string(exist)}))
500         cout << "Producto registrado correctamente.\n";
501 }
502
503 static void modificarProducto() {
504     titulo("Modificar producto");
505     listarProductos();
506     string codigo = leerObligatorio("Código del producto: ");
507     Tabla t;
508     ejecutar("SELECT nombre, costo, precio FROM Producto WHERE codigo = ?", {codigo}, &t);
509     if (t.empty()) {
510         cout << "No existe un producto con ese código.\n";
511         return;
512     }
513     cout << "(Presione Enter para conservar el valor actual)\n";
514     string nombre = leerOpcional("Nombre      ", t[0][0]);
515     double costo = stod(t[0][1]), precio = stod(t[0][2]);
516     string c = leer("Costo (Q) [" + dinero(t[0][1]) + "]: ");
517     string p = leer("Precio (Q) [" + dinero(t[0][2]) + "]: ");
518     try {
519         if (!c.empty()) costo = stod(c);
520         if (!p.empty()) precio = stod(p);
521     } catch (...) {
522         cout << "Valor numérico inválido. No se realizaron cambios.\n";
523         return;
524     }
525     if (costo < 0 || precio < 0) {
526         cout << "Los valores no pueden ser negativos. No se realizaron cambios.\n";
527         return;
528     }
529     if (ejecutar("UPDATE Producto SET nombre=?, costo=?, precio=? WHERE codigo=?",
530                  {nombre, num2(costo), num2(precio), codigo}))
531         cout << "Producto actualizado correctamente.\n";
532 }
533
534 static void reabastecerProducto() {
535     titulo("Ingreso de mercadería (inventario)");
536     listarProductos();
537     string codigo = leerObligatorio("Código del producto: ");
538     Tabla t;
539     ejecutar("SELECT nombre, existencia FROM Producto WHERE codigo = ?", {codigo}, &t);
540     if (t.empty()) {
541         cout << "No existe un producto con ese código.\n";
542         return;
543     }
544     int cant = leerEntero("Cantidad que ingresa a bodega: ", 1, INT_MAX);
545     if (ejecutar("UPDATE Producto SET existencia = existencia + ? WHERE codigo = ?", {to_string(cant), codigo}))
546         cout << "Existencia actualizada: " << t[0][0] << " ahora tiene " << (stoi(t[0][1]) + cant) << " unidades.\n";
547 }
548
549 // ============================================================================
550 //  6. MÓDULO DE FACTURACIÓN
551 // ============================================================================
552 static void mostrarFactura(const string& correlativo) {
553     Tabla f;
```

Anexo A. Código fuente: main.cpp (C++17 + SQLite3)

```
    ejecutar("SELECT f.correlativo, f.serie, f.fecha, c.nombre, c.nit, e.nombre, f.condicion, f.plazo, "
             "f.fecha_vencimiento, f.subtotal, f.iva, f.total "
             "FROM Factura f JOIN Cliente c ON c.id = f.cliente_id JOIN Empleado e ON e.codigo = f.vendedor_codigo "
             "WHERE f.correlativo = ?",
             {correlativo}, &f);
    if (f.empty()) {
        cout << "No existe la factura " << correlativo << ".\n";
        return;
    }
    const Fila& r = f[0];
    cout << "\n==================== FACTURA ====================\n";
    cout << "No. correlativo : " << r[0] << "     Serie: " << r[1] << "\n";
    cout << "Fecha           : " << r[2] << "\n";
    cout << "Cliente         : " << r[3] << "   (NIT " << r[4] << ")\n";
    cout << "Vendedor        : " << r[5] << "\n";
    cout << "Condición       : " << r[6];
    if (r[6] == "Crédito") cout << "  (plazo " << r[7] << " días, vence " << r[8] << ")";
    cout << "\n";
    Tabla d;
    ejecutar("SELECT d.producto_codigo, p.nombre, d.cantidad, d.precio_unitario, d.subtotal "
             "FROM DetalleFactura d JOIN Producto p ON p.codigo = d.producto_codigo "
             "WHERE d.factura_correlativo = ? ORDER BY d.id",
             {correlativo}, &d);
    for (Fila& x : d) {
        x[3] = dinero(x[3]);
        x[4] = dinero(x[4]);
    }
    imprimirTabla({"Código", "Producto", "Cant.", "P. unit.", "Subtotal"}, {8, 26, 6, 10, 12}, d,
                  {false, false, true, true, true});
    cout << "Subtotal : Q " << dinero(r[9]) << "\n";
    cout << "IVA 12%  : Q " << dinero(r[10]) << "\n";
    cout << "TOTAL    : Q " << dinero(r[11]) << "\n";
    cout << "Saldo    : Q " << dinero(saldoFactura(correlativo)) << "\n";
    cout << "================================================\n";
}

struct Linea {
    string codigo, nombre;
    int cantidad;
    double precio;
};

static void generarFactura() {
    titulo("Generar factura");
    listarClientes(rolActual == VENDEDOR);
    int cid = leerEntero("ID del cliente: ", 1, INT_MAX);
    Tabla c;
    ejecutar("SELECT id, nombre, vendedor_codigo FROM Cliente WHERE id = ?", {to_string(cid)}, &c);
    if (c.empty()) {
        cout << "Cliente no encontrado.\n";
        return;
    }
    if (rolActual == VENDEDOR && c[0][2] != usuarioActual) {
        cout << "Ese cliente no pertenece a su cartera. No puede facturarle.\n";
        return;
    }
    cout << "Cliente: " << c[0][1] << "  |  Vendedor asignado: " << c[0][2] << "\n";

    int cond = leerEntero("Condición (1=Contado, 2=Crédito): ", 1, 2);
    int plazo = 0;
    if (cond == 2) plazo = leerEntero("Plazo en días (1-365): ", 1, 365);

    cout << "\nProductos disponibles:\n";
    listarProductos();

    vector<Linea> lineas;
    while (true) {
        string cod = leer("Código de producto (Enter para terminar): ");
        if (cod.empty()) break;
        Tabla p;
        ejecutar("SELECT codigo, nombre, precio, existencia FROM Producto WHERE codigo = ?", {cod}, &p);
        if (p.empty()) {
            cout << "  Producto no encontrado.\n";
            continue;
        }
        int yaPedido = 0;
        for (const Linea& l : lineas)
            if (l.codigo == p[0][0]) yaPedido += l.cantidad;
        int disponible = stoi(p[0][3]) - yaPedido;
```

Anexo A. Código fuente: main.cpp (C++17 + SQLite3)

```
633         if (disponible <= 0) {
634             cout << "  Sin existencia disponible de " << p[0][1] << ".\n";
635             continue;
636         }
637         int cant = leerEntero("Cantidad (disponible " + to_string(disponible) + "): ", 1, INT_MAX);
638         if (cant > disponible) {
639             cout << "  Existencia insuficiente. Solo hay " << disponible << " unidades disponibles.\n";
640             continue;
641         }
642         lineas.push_back({p[0][0], p[0][1], cant, stod(p[0][2])});
643         cout << "  Agregado: " << cant << " x " << p[0][1] << " a Q " << dinero(p[0][2]) << " c/u\n";
644     }
645     if (lineas.empty()) {
646         cout << "No se agregó ningún producto. Factura cancelada.\n";
647         return;
648     }
649 
650     double subtotal = 0;
651     for (const Linea& l : lineas) subtotal += redondear(l.cantidad * l.precio);
652     subtotal = redondear(subtotal);
653     double iva = redondear(subtotal * TASA_IVA);
654     double total = redondear(subtotal + iva);
655 
656     cout << "\nResumen: Subtotal Q " << num2(subtotal) << " | IVA Q " << num2(iva) << " | TOTAL Q " << num2(total) <<
657     if (!confirmar("¿Confirmar la factura?")) {
658         cout << "Factura cancelada.\n";
659         return;
660     }
661 
662     // Todo se guarda en una transacción: o se guarda completo o no se guarda nada.
663     ejecutar("BEGIN");
664     bool ok = ejecutar("INSERT INTO Factura(serie,fecha,cliente_id,vendedor_codigo,condicion,plazo,fecha_vencimiento,s
665                        "VALUES('A', date('now','localtime'), ?, ?, ?, ?, date('now','localtime','+' || ? || ' days'),
666                        {to_string(cid), c[0][2], cond == 1 ? "Contado" : "Crédito", to_string(plazo), to_string(plazo)
667                         num2(subtotal), num2(iva), num2(total)});
668     string correlativo = to_string((long long)sqlite3_last_insert_rowid(db));
669 
670     for (size_t i = 0; ok && i < lineas.size(); i++) {
671         const Linea& l = lineas[i];
672         ok = ejecutar("INSERT INTO DetalleFactura(factura_correlativo,producto_codigo,cantidad,precio_unitario,subtota
673                       {correlativo, l.codigo, to_string(l.cantidad), num2(l.precio), num2(redondear(l.cantidad * l.pre
674         if (ok) {
675             ok = ejecutar("UPDATE Producto SET existencia = existencia - ? WHERE codigo = ? AND existencia >= ?",
676                           {to_string(l.cantidad), l.codigo, to_string(l.cantidad)});
677             if (ok && sqlite3_changes(db) != 1) ok = false;
678         }
679     }
680     if (ok && cond == 1)  // Contado: un solo pago por el total
681         ok = ejecutar("INSERT INTO Pago(factura_correlativo,fecha,monto,saldo) VALUES(?, date('now','localtime'), ?, 0
682                       {correlativo, num2(total)});
683 
684     if (ok) {
685         ejecutar("COMMIT");
686         cout << "\nFactura generada correctamente.\n";
687         mostrarFactura(correlativo);
688     } else {
689         ejecutar("ROLLBACK");
690         cout << "No se pudo generar la factura. No se realizó ningún cambio.\n";
691     }
692 }
693 
694 static void consultarFactura() {
695     titulo("Consultar factura");
696     Tabla t;
697     string sql = "SELECT f.correlativo, f.fecha, c.nombre, f.condicion, f.total FROM Factura f "
698                  "JOIN Cliente c ON c.id = f.cliente_id";
699     Fila p;
700     if (rolActual == VENDEDOR) {
701         sql += " WHERE f.vendedor_codigo = ?";
702         p.push_back(usuarioActual);
703     }
704     sql += " ORDER BY f.correlativo";
705     ejecutar(sql, p, &t);
706     for (Fila& r : t) r[4] = dinero(r[4]);
707     imprimirTabla({"Factura", "Fecha", "Cliente", "Condición", "Total"}, {8, 10, 24, 10, 12}, t,
708                   {true, false, false, false, true});
709     if (t.empty()) return;
710     int n = leerEntero("Número de factura a ver (0 para volver): ", 0, INT_MAX);
711     if (n == 0) return;
```

Anexo A. Código fuente: main.cpp (C++17 + SQLite3)

```
    // un vendedor solo puede ver sus propias facturas
    bool permitido = false;
    for (const Fila& r : t)
        if (r[0] == to_string(n)) permitido = true;
    if (!permitido) {
        cout << "Factura no encontrada.\n";
        return;
    }
    mostrarFactura(to_string(n));
}

// ============================================================================
//  7. MÓDULO DE PAGOS
// ============================================================================
static void registrarPago() {
    titulo("Registrar pago");
    Tabla t;
    string sql = "SELECT f.correlativo, f.fecha, c.nombre, f.condicion, f.total, "
                 "ROUND(f.total - COALESCE((SELECT SUM(monto) FROM Pago WHERE factura_correlativo=f.correlativo),0),2) "
                 "FROM Factura f JOIN Cliente c ON c.id = f.cliente_id WHERE saldo > 0";
    Fila p;
    if (rolActual == VENDEDOR) {
        sql += " AND f.vendedor_codigo = ?";
        p.push_back(usuarioActual);
    }
    sql += " ORDER BY f.correlativo";
    ejecutar(sql, p, &t);
    if (t.empty()) {
        cout << "No hay facturas con saldo pendiente.\n";
        return;
    }
    for (Fila& r : t) {
        r[4] = dinero(r[4]);
        r[5] = dinero(r[5]);
    }
    cout << "Facturas con saldo pendiente:\n";
    imprimirTabla({"Factura", "Fecha", "Cliente", "Condición", "Total", "Saldo"}, {8, 10, 22, 10, 12, 12}, t,
                  {true, false, false, false, true, true});

    string corr = leerObligatorio("Número de factura a abonar: ");
    string saldoTxt;
    for (const Fila& r : t)
        if (r[0] == corr) saldoTxt = r[5];
    if (saldoTxt.empty()) {
        cout << "Esa factura no tiene saldo pendiente o no está disponible para usted.\n";
        return;
    }
    double saldo = stod(saldoTxt);
    cout << "Saldo pendiente: Q " << num2(saldo) << "\n";
    double monto = leerDecimal("Monto del pago (Q): ", 0.01);
    monto = redondear(monto);
    if (monto > saldo + 0.001) {
        cout << "El monto (Q " << num2(monto) << ") excede el saldo pendiente. Pago no registrado.\n";
        return;
    }
    double nuevo = redondear(saldo - monto);
    if (ejecutar("INSERT INTO Pago(factura_correlativo,fecha,monto,saldo) VALUES(?, date('now','localtime'), ?, ?)",
                 {corr, num2(monto), num2(nuevo)})) {
        cout << "Pago registrado. Saldo restante de la factura " << corr << ": Q " << num2(nuevo) << "\n";
        if (nuevo == 0) cout << "La factura quedó cancelada en su totalidad.\n";
    }
}

static void consultarPagos() {
    titulo("Pagos de una factura");
    int n = leerEntero("Número de factura: ", 1, INT_MAX);
    Tabla f;
    string sql = "SELECT correlativo FROM Factura WHERE correlativo = ?";
    Fila p = {to_string(n)};
    if (rolActual == VENDEDOR) {
        sql += " AND vendedor_codigo = ?";
        p.push_back(usuarioActual);
    }
    ejecutar(sql, p, &f);
    if (f.empty()) {
        cout << "Factura no encontrada.\n";
        return;
    }
    Tabla t;
```

Anexo A. Código fuente: main.cpp (C++17 + SQLite3)

```
    ejecutar("SELECT correlativo, fecha, monto, saldo FROM Pago WHERE factura_correlativo = ? ORDER BY correlativo",
             {to_string(n)}, &t);
    for (Fila& r : t) {
        r[2] = dinero(r[2]);
        r[3] = dinero(r[3]);
    }
    imprimirTabla({"Pago", "Fecha", "Monto", "Saldo"}, {6, 10, 12, 12}, t, {true, false, true, true});
    cout << "Saldo actual de la factura: Q " << dinero(saldoFactura(to_string(n))) << "\n";
}

// ============================================================================
//  8. MÓDULO DE REPORTES
// ============================================================================
static void reporteClientes() {
    titulo("REPORTE DE CLIENTES");
    Tabla t;
    ejecutar("SELECT id, nombre, nit, telefono, contacto, vendedor_codigo, direccion, email, web FROM Cliente ORDER BY
    Tabla principal;
    vector<string> detalle;
    for (const Fila& r : t) {
        principal.push_back({r[0], r[1], r[2], r[3], r[4], r[5]});
        string d = "Dirección: " + r[6];
        if (!r[7].empty()) d += "  |  E-mail: " + r[7];
        if (!r[8].empty()) d += "  |  Web: " + r[8];
        detalle.push_back(d);
    }
    imprimirTabla({"ID", "Nombre", "NIT", "Teléfono", "Contacto", "Vendedor"}, {4, 24, 11, 10, 18, 8}, principal,
                  {true}, detalle);
}

static void reporteVendedores() {
    titulo("REPORTE DE VENDEDORES");
    Tabla t;
    ejecutar("SELECT e.codigo, e.dpi, e.nombre, e.telefono, e.email, "
             "(SELECT COUNT(*) FROM Cliente c WHERE c.vendedor_codigo = e.codigo), e.direccion "
             "FROM Empleado e WHERE e.tipo = 'Vendedor' ORDER BY e.codigo",
             {}, &t);
    Tabla principal;
    vector<string> detalle;
    for (const Fila& r : t) {
        principal.push_back({r[0], r[1], r[2], r[3], r[4], r[5]});
        detalle.push_back("Dirección: " + r[6]);
    }
    imprimirTabla({"Código", "DPI", "Nombre", "Teléfono", "E-mail", "Clientes"}, {7, 13, 22, 9, 22, 8}, principal,
                  {false, false, false, false, false, true}, detalle);
}

static void reporteProductosVendidos() {
    titulo("REPORTE DE PRODUCTOS VENDIDOS");
    string desde = leer("Desde (AAAA-MM-DD, Enter = sin límite): ");
    string hasta = leer("Hasta (AAAA-MM-DD, Enter = sin límite): ");
    Tabla t;
    ejecutar("SELECT d.producto_codigo, p.nombre, SUM(d.cantidad), SUM(d.subtotal) "
             "FROM DetalleFactura d JOIN Producto p ON p.codigo = d.producto_codigo "
             "JOIN Factura f ON f.correlativo = d.factura_correlativo "
             "WHERE (? = '' OR f.fecha >= ?) AND (? = '' OR f.fecha <= ?) "
             "GROUP BY d.producto_codigo, p.nombre ORDER BY SUM(d.cantidad) DESC",
             {desde, desde, hasta, hasta}, &t);
    for (Fila& r : t) r[3] = dinero(r[3]);
    imprimirTabla({"Código", "Producto", "Unidades", "Monto vendido"}, {8, 28, 9, 14}, t, {false, false, true, true});
}

static void reporteVentasDelDia() {
    titulo("LISTADO DE VENTAS (FACTURAS DEL DÍA)");
    Tabla t;
    ejecutar("SELECT f.correlativo, f.serie, c.nombre, e.nombre, f.condicion, f.subtotal, f.iva, f.total "
             "FROM Factura f JOIN Cliente c ON c.id = f.cliente_id JOIN Empleado e ON e.codigo = f.vendedor_codigo "
             "WHERE f.fecha = date('now','localtime') ORDER BY f.correlativo",
             {}, &t);
    double suma = 0;
    for (Fila& r : t) {
        suma += stod(r[7]);
        r[5] = dinero(r[5]);
        r[6] = dinero(r[6]);
        r[7] = dinero(r[7]);
    }
    imprimirTabla({"Factura", "Serie", "Cliente", "Vendedor", "Condición", "Subtotal", "IVA", "Total"},
                  {8, 5, 22, 22, 10, 10, 9, 11}, t, {true, false, false, false, false, true, true, true});
    cout << "TOTAL VENDIDO DEL DÍA: Q " << num2(suma) << "\n";
```

Anexo A. Código fuente: main.cpp (C++17 + SQLite3)

```
}

static void reportePagosDelDia() {
    titulo("REPORTE DE PAGOS DEL DÍA");
    Tabla t;
    ejecutar("SELECT p.correlativo, p.factura_correlativo, c.nombre, f.condicion, p.monto, p.saldo "
             "FROM Pago p JOIN Factura f ON f.correlativo = p.factura_correlativo JOIN Cliente c ON c.id = f.cliente_i
             "WHERE p.fecha = date('now','localtime') ORDER BY p.correlativo",
             {}, &t);
    double suma = 0;
    for (Fila& r : t) {
        suma += stod(r[4]);
        r[4] = dinero(r[4]);
        r[5] = dinero(r[5]);
    }
    imprimirTabla({"Pago", "Factura", "Cliente", "Condición", "Monto", "Saldo"}, {6, 8, 24, 10, 12, 12}, t,
                  {true, true, false, false, true, true});
    cout << "TOTAL RECIBIDO DEL DÍA: Q " << num2(suma) << "\n";
}

static void reporteInventario() {
    titulo("REPORTE DE INVENTARIO EXISTENTE");
    Tabla t;
    ejecutar("SELECT codigo, nombre, costo, precio, existencia FROM Producto ORDER BY codigo", {}, &t);
    for (Fila& r : t) {
        r[2] = dinero(r[2]);
        r[3] = dinero(r[3]);
    }
    imprimirTabla({"Código", "Producto", "Costo", "Precio", "Existencia"}, {8, 28, 10, 10, 10}, t,
                  {false, false, true, true, true});
}

// ============================================================================
//  9. MENÚS Y SESIÓN
// ============================================================================
struct Opcion {
    string texto;
    void (*accion)();
};

static void correrMenu(const string& nombre, const vector<Opcion>& ops, const string& textoSalida) {
    while (true) {
        cout << "\n========== " << nombre << " ==========\n";
        for (size_t i = 0; i < ops.size(); i++) cout << "  " << (i + 1) << ". " << ops[i].texto << "\n";
        cout << "  0. " << textoSalida << "\n";
        int o = leerEntero("Seleccione una opción: ", 0, (int)ops.size());
        if (o == 0) return;
        ops[o - 1].accion();
    }
}

static void menuEmpleados() {
    correrMenu("EMPLEADOS", {{"Registrar empleado", registrarEmpleado},
                             {"Modificar empleado", modificarEmpleado},
                             {"Consultar empleados", []() { titulo("Empleados registrados"); listarEmpleados(); }}},
               "Volver");
}
static void menuClientes() {
    correrMenu("CLIENTES", {{"Registrar cliente", registrarCliente},
                            {"Modificar cliente", modificarCliente},
                            {"Consultar clientes", consultarClientes},
                            {"Historial de un cliente (facturas y pagos)", historialCliente}},
               "Volver");
}
static void menuProductos() {
    correrMenu("PRODUCTOS E INVENTARIO", {{"Registrar producto", registrarProducto},
                                          {"Modificar producto", modificarProducto},
                                          {"Ingreso de mercadería (aumentar existencia)", reabastecerProducto},
                                          {"Consultar productos", []() { titulo("Productos"); listarProductos(); }}},
               "Volver");
}
static void menuFacturacion() {
    correrMenu("FACTURACIÓN", {{"Generar factura", generarFactura}, {"Consultar factura", consultarFactura}}, "Volver"
}
static void menuPagos() {
    correrMenu("PAGOS", {{"Registrar pago", registrarPago}, {"Consultar pagos de una factura", consultarPagos}}, "Volv
}
static void menuReportes() {
    correrMenu("REPORTES", {{"Reporte de Clientes", reporteClientes},
```

Anexo A. Código fuente: main.cpp (C++17 + SQLite3)

```
                            {"Reporte de Vendedores", reporteVendedores},
                            {"Reporte de Productos Vendidos", reporteProductosVendidos},
                            {"Listado de Ventas (Facturas del Día)", reporteVentasDelDia},
                            {"Reporte de Pagos del Día", reportePagosDelDia},
                            {"Reporte de Inventario Existente", reporteInventario}},
                "Volver");
}

static bool iniciarSesion() {
    cout << "=====================================================\n";
    cout << "  SISTEMA DE FACTURACIÓN - REPRESENTACIONES \"CHAPINA\"\n";
    cout << "=====================================================\n";
    for (int intento = 1; intento <= 3; intento++) {
        string u = leer("Usuario (admin o código de empleado): ");
        string c = leer("Clave: ");
        if (u == "admin" && c == "admin123") {
            rolActual = ADMIN;
            usuarioActual = "admin";
            nombreUsuario = "Administrador";
            return true;
        }
        Tabla t;
        ejecutar("SELECT nombre FROM Empleado WHERE codigo = ? AND dpi = ? AND tipo = 'Vendedor'", {u, c}, &t);
        if (!t.empty()) {
            rolActual = VENDEDOR;
            usuarioActual = u;
            nombreUsuario = t[0][0];
            return true;
        }
        cout << "Credenciales incorrectas (intento " << intento << " de 3).\n";
    }
    return false;
}

int main(int argc, char** argv) {
#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);  // para mostrar tildes y ñ correctamente en la consola de Windows
    SetConsoleCP(CP_UTF8);
#endif
    string ruta = (argc > 1) ? argv[1] : "chapina.db";
    if (sqlite3_open(ruta.c_str(), &db) != SQLITE_OK) {
        cerr << "No se pudo abrir la base de datos: " << sqlite3_errmsg(db) << "\n";
        return 1;
    }
    crearEsquema();

    if (!iniciarSesion()) {
        cout << "Demasiados intentos fallidos. El sistema se cerrará.\n";
        sqlite3_close(db);
        return 1;
    }
    if (rolActual == ADMIN) cout << "\nBienvenido(a), Administrador\n";
    else cout << "\nBienvenido(a), " << nombreUsuario << " (Vendedor)\n";

    if (rolActual == ADMIN) {
        correrMenu("MENÚ PRINCIPAL", {{"Empleados", menuEmpleados},
                                      {"Clientes", menuClientes},
                                      {"Productos e Inventario", menuProductos},
                                      {"Facturación", menuFacturacion},
                                      {"Pagos", menuPagos},
                                      {"Reportes", menuReportes}},
                   "Salir del sistema");
    } else {
        correrMenu("MENÚ PRINCIPAL", {{"Clientes", menuClientes}, {"Facturación", menuFacturacion}, {"Pagos", menuPago
                   "Salir del sistema");
    }
    cout << "\nSesión finalizada. ¡Hasta luego!\n";
    sqlite3_close(db);
    return 0;
}
```